Journal of King Saud University Computer and Information Sciences (2026)38:932
https://doi.org/10.1007/s44443-026-01325-x

RESEARCH

MQTT-HS: A formally verified and proxy-mediated hybrid security protocol for MQTT-based IoT networks

Manar Almalki[1] · Majid Alshammari[1]

Received: 6 July 2026 / Accepted: 29 August 2026
© The Author(s) 2026

Abstract
MQTT enables efficient communication in resource-constrained Internet of Things (IoT) systems but does not provide integrated protection against cryptographic and behavioral threats. This paper presents MQTT-HS, a proxy-mediated security protocol that combines certificate-based authentication, ephemeral ECDH/HKDF session establishment, AES-256-GCM payload protection, controlled key refresh, and metadata-based hierarchical intrusion detection with inline enforcement. ProVerif verified the modeled secrecy, authentication, replay-resistance, message-flow agreement, and epoch-key secrecy properties of the cryptographic core. MQTT-HS supports forward secrecy across expired epochs through fresh ephemeral ECDH material, independent epoch-key derivation, and secure erasure of obsolete key material. An ESP32 implementation delivered all 360 protected messages and completed six in-session rekeys, with an average AES-256-GCM encryption latency of 0.354 ms per message. Controlled runtime experiments rejected all evaluated replay, untrusted-certificate, and invalid-signature attempts. The hierarchical XGBoost IDS achieved balanced accuracies of 0.808, 0.960, and 0.862 across its three stages, although Stage 0 produced a 37.5% false-positive rate on the limited legitimate holdout subset. Compared with the evaluated mutual TLS baselines, MQTT-HS preserved more endpoint free heap and provided MQTT-aware detection and enforcement, while requiring greater network bandwidth and a trusted Security Proxy. The results support the feasibility of MQTT-HS under the evaluated prototype configuration.

Keywords MQTT security · Proxy-mediated security · Hierarchical ML-IDS · Intrusion detection · Formal verification · AES-GCM

1 Introduction

The Internet of Things (IoT) has become a core component of modern digital infrastructures, enabling a wide range of devices to exchange data across heterogeneous networks to support critical applications Choudhary (2024); Chui et al. (2023); Salih et al. (2025). However, the decentralized and resource-constrained nature of IoT also increases its exposure to cyber threats. Weak configurations, outdated firmware, default credentials, and insecure communication channels expose IoT devices to compromise and botnet-driven denial-of-service attacks Sahota & Vlajic (2021); Sasi et al. (2024).

Message Queuing Telemetry Transport (MQTT) is a lightweight machine-to-machine communication protocol widely used in constrained IoT environments Al Hanif & Ilyas (2024). MQTT follows a publish/subscribe model in which Publishers send messages to topics through a Broker and Subscribers receive messages from their subscribed topics Im & Lim (2023). This model decouples data producers from consumers and supports asynchronous communication with compact packets, low bandwidth consumption, and configurable Quality of Service (QoS) levels Aloufi & Alhazmi (2020); Mishra & Kertesz (2020).

Despite its efficiency, MQTT does not natively provide payload encryption, strong mutual authentication, or message-integrity verification. Authentication is commonly implemented using username and password fields in the CONNECT packet, which may be exposed if transport-layer protection is not applied Dinculeană & Cheng (2019);

✉ Majid Alshammari
m.alshammari@tu.edu.sa
Manar Almalki
s44680292@students.tu.edu.sa

[1] Taif University, Ta'if, Saudi Arabia

Published online: 19 October 2026

Guşiţă et al. (2025). Both MQTT v3.1.1 and MQTT v5.0 still rely mainly on external mechanisms, such as Transport Layer Security (TLS), for confidentiality and integrity OASIS Open (2019, 2015). Consequently, MQTT-based systems remain vulnerable to eavesdropping, spoofing, brute-force authentication attempts, malformed packet injection, flooding, replay attacks, and denial-of-service attacks Dinculeană & Cheng (2019); Husnain et al. (2022); Kant et al. (2021).

TLS is commonly used to secure MQTT traffic. TLS provides transport-layer confidentiality and integrity but introduces handshake, certificate-processing, memory, and communication costs on constrained devices Alkhafajee et al. (2021); Gavriilidis et al. (2025); Prantl et al. (2021). Moreover, TLS protects the communication channel but remains unaware of MQTT-specific behavioral misuse, such as flooding, brute-force attempts, broker abuse, malformed packet sequences, and slow-rate denial-of-service behavior.

Machine learning-based Intrusion Detection Systems (IDSs) analyze communication patterns, packet sizes, message frequency, timing intervals, and protocol behavior to identify anomalous IoT traffic Bankó et al. (2025); Reis (2025). However, some IDS approaches require plaintext payload inspection, while others operate as offline or passive monitors without active enforcement in the MQTT workflow Alashjaee & Alqahtani (2025); Khan et al. (2021); Kikissagbe & Adda (2024). Conversely, cryptographic protocols protect confidentiality and authentication but do not directly detect behavioral attacks against the Broker or MQTT control flow. This separation leaves a gap for a unified MQTT-aware design that combines verified key establishment, authenticated payload protection, metadata-only runtime detection, and inline enforcement before Broker admission.

To address this gap, this paper proposes MQTT-HS, a proxy-mediated hybrid security protocol for MQTT-based IoT communication. MQTT-HS combines ECDH/HKDF-based session establishment, AES-256-GCM authenticated encryption, controlled key refresh, and a three-stage hierarchical ML-IDS. The ML-IDS analyzes protocol-safe structural metadata through sequential sliding windows without using plaintext payload content and integrates its decisions into inline enforcement before Broker admission. Payload confidentiality is maintained outside the trusted Proxy boundary.

The main contributions of this paper are summarized as follows:

1. We propose MQTT-HS, a proxy-mediated MQTT security protocol that integrates ECDH/HKDF-based session establishment, AES-256-GCM authenticated encryption, controlled key refresh, and Proxy-side enforcement before Broker admission.
2. We design a hierarchical ML-IDS that detects MQTT attacks from protocol-safe structural metadata using sequential sliding windows and integrates its decisions into the Security Proxy for inline enforcement without using plaintext payload content.
3. We formally analyze the cryptographic core of MQTT-HS using ProVerif, verifying modeled secrecy, authentication, replay resistance, message-flow agreement, and epoch-key secrecy across key refresh. MQTT-HS supports forward secrecy across expired epochs through fresh ephemeral ECDH material, independent epoch-key derivation, and secure erasure of obsolete key material.
4. We evaluate MQTT-HS through host and ESP32 implementations, mutual TLS baselines, endpoint and Proxy resource measurements, packet-level network analysis, and controlled behavioral and protocol-level attack scenarios.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 introduces the system model and preliminaries. Sections 4 and 5 detail the proposed MQTT-HS protocol and the privacy-preserving ML-IDS, respectively. Sections 6 and 7 present the formal security analysis and experimental evaluation. Section 8 discusses the findings and comparison with related approaches, and Section 9 concludes the paper.

2 Related work

MQTT has become one of the most widely adopted messaging protocols in IoT environments because of its lightweight publish/subscribe model, low communication overhead, and suitability for constrained devices. However, its minimal design leaves important security responsibilities to external mechanisms, including authentication, encryption, access control, anomaly detection, and broker-side enforcement. Existing research has therefore addressed MQTT security from several directions: vulnerability analysis and conventional transport-layer protection, lightweight cryptographic and formally verified protocols, machine learning-based intrusion detection, and hybrid crypto-AI security designs. Table 1 summarizes the main research directions and the functional scope reported by representative studies.

Table 1 Summary of representative MQTT security research directions

Research Direction	Ref.	Main Focus	Contribution	Limitation
Ecosystem Analysis	Laghari et al. (2024)	MQTT security weaknesses (auth, brokers, traffic).	Maps broad attack surfaces beyond plaintext data.	Analytical only; lacks deployable security protocol.
Transport Protection	Seoane et al. (2021)	TLS/DTLS in constrained MQTT/ CoAP nodes.	Evaluates performance cost (latency, bandwidth).	Cannot detect behavioral attacks (e.g., SlowITe).
Lightweight Crypto	Voloshyn et al. (2024)	Authenticated encryption for IoT.	Reduces overhead while preserving data integrity.	Lacks dynamic keys, ML-IDS, and inline blocking.
Layered Encryption	Alharbi et al. (2025)	Multi-layer edge-to-cloud encryption.	Separates edge encryption from cloud protection.	No integrated metadata ML-IDS or proxy enforcement.
ECC Authentication	Yusoff et al. (2024)	ECC-based auth for smart-home MQTT.	Low-overhead cryptographic authentication.	Lacks AES-GCM payloads and hierarchical IDS.
Formal Security	Gao et al. (2025)	Verification of MQTT crypto properties.	Provides formal modeling and provable security.	Omits runtime behavioral ML anomaly detection.
Ensemble ML-IDS	Siddharthan et al. (2022)	MQTT-specific traffic feature detection.	Demonstrates the utility of protocol-aware ML features.	Purely detection-oriented; lacks crypto integration.
Feature Engineering	Imran et al. (2024)	Optimized ML classification for MQTT.	Reports strong classification performance using engineered features.	Environment-specific; no active inline blocking.
IDS Datasets	Aqachtoul et al. (2025)	MQTTEEB-D benchmark development.	Realistic dataset for DoS, brute force, etc.	Dataset contribution only; no proposed protocol.
ML vs. DL Threat Detection	Alhossainy & Malekzadeh (2026)	Model evaluation using MQTTEEB-D.	Reports competitive performance for classical ML models.	Offline detection; lacks broker-side enforcement.
Hybrid Crypto-ML	Karimunda et al. (2025)	ECC protection + ANN anomaly detection.	Combines cryptography with intelligent IDS.	Lacks formal verification and metadata-only mode.
Encrypted Traffic ML	Qasem et al. (2026)	ML detection over encrypted payloads.	Maintains IDS efficacy on protected channels.	Cloud-oriented detection rather than inline proxy.
Proposed Protocol	**MQTT-HS**	**Proxy-mediated hybrid security**.	**Integrates ECDH/AES-GCM, ML-IDS & formal verification**.	**Requires a trusted Proxy and sliding-window warm-up**.

2.1 MQTT security challenges and conventional protection

A significant body of MQTT security research has focused on analyzing the protocol ecosystem and its deployment weaknesses. Laghari et al. (2024) showed that MQTT threats are not limited to plaintext transmission, but also include weak authentication, unauthorized broker access, topic misuse, malformed traffic, denial-of-service behavior, broker software weaknesses, and deployment misconfigurations. These findings indicate that MQTT security should not be treated as a single-layer encryption problem, but as a broader protocol-level and deployment-level challenge.

Conventional protection mechanisms, particularly TLS-based MQTT deployments, provide confidentiality and integrity at the transport layer. However, empirical evaluations show that TLS introduces additional latency, bandwidth consumption, and CPU overhead, which can be problematic for constrained IoT devices and lossy network conditions Seoane et al. (2021). More importantly, TLS is not aware of MQTT control behavior and therefore cannot directly detect protocol-level misuse such as flooding, SlowITe, brute-force connection attempts, broker abuse, or malformed MQTT packet sequences. This limitation motivates security protocols that go beyond channel protection by combining data confidentiality with behavioral monitoring and active traffic control before broker admission.

2.2 Lightweight cryptography, authentication, and formal verification

A second research direction focuses on lightweight cryptographic mechanisms for MQTT and resource-constrained IoT environments. Lightweight authenticated encryption schemes, including NIST-oriented lightweight ciphers, have been investigated to reduce the overhead of traditional security mechanisms while preserving confidentiality and integrity Voloshyn et al. (2024). These studies provide evidence that lightweight cryptographic mechanisms can be applied to MQTT-based IoT communication. However, many of them remain cipher-centric and do not cover the full security lifecycle, including dynamic key establishment, key refresh, broker-aware enforcement, and runtime intrusion detection.

Other studies have proposed layered or broker-assisted encryption designs. For example, lightweight edge encryption has been combined with stronger cloud-side encryption through bridge or multi-broker MQTT settings Alharbi et al. (2025). Such approaches recognize that constrained edge devices and cloud-facing components have different security requirements, but they remain primarily encryption-oriented. Similarly, authentication-focused approaches have adopted elliptic-curve cryptography (ECC) to strengthen MQTT-based systems while reducing the cost of public-key operations compared with heavier alternatives Yusoff et al. (2024). Nevertheless, these approaches generally do not integrate behavioral intrusion detection or inline blocking decisions into the MQTT communication workflow.

Formal verification is another important direction for secure MQTT protocol design. Formalized MQTT security protocols have been proposed to verify properties such as authentication, key secrecy, replay resistance, and message integrity Gao et al. (2025). These works strengthen confidence in the cryptographic correctness of the protocol interactions. However, they typically focus on secrecy and authentication properties and do not address runtime MQTT traffic anomalies, ML-based attack classification, or active proxy-level enforcement. MQTT-HS builds on this direction by combining ECDH/HKDF-based session establishment, AES-GCM authenticated encryption, ProVerif-based verification, controlled key refresh, and runtime IDS-assisted enforcement within a single MQTT-aware security protocol.

2.3 Machine learning-based MQTT intrusion detection

Another major research direction focuses on MQTT intrusion detection using machine learning and traffic metadata. MQTT-specific IDS datasets and testbeds have shown that attacks can often be detected using traffic-level and protocol-level features, such as packet timing, packet length, TCP flags, MQTT control flags, QoS values, and connection behavior Siddharthan et al. (2022). Feature engineering studies on MQTT datasets have also shown that protocol-aware features extracted from PCAP traces can improve classification performance Imran et al. (2024). However, over-reliance on static or environment-specific identifiers, such as source addresses or deployment-specific attributes, may reduce generalizability and limit transferability to other MQTT environments.

The availability of more realistic datasets, particularly MQTTEEB-D, has further advanced MQTT intrusion detection research Aqachtoul et al. (2025). MQTTEEB-D includes MQTT attack categories such as DoS, SlowITe, malformed data injection, brute force, and publish flooding, and captures timing and traffic variations from a real-time IoT testbed. Recent evaluations on this dataset indicate that classical ML models can achieve competitive detection performance relative to the evaluated deep-learning models Alhossainy & Malekzadeh (2026). This motivates the evaluation of compact classical models for latency- and resource-aware MQTT intrusion detection.

Despite these advancements, most MQTT IDS studies remain detection-oriented. They optimize classification accuracy, feature selection, or model efficiency, but often do not specify how predictions are enforced within the MQTT communication workflow. In addition, many IDS solutions operate as offline or passive monitors and do not provide cryptographic session establishment, key lifecycle management, formal protocol verification, or inline blocking before broker admission. MQTT-HS addresses this limitation by embedding metadata-only sliding-window detection into the Security Proxy, allowing malicious MQTT behavior to be classified and blocked before it reaches the broker.

2.4 Hybrid crypto-AI frameworks and the positioning of MQTT-HS

Recent research has begun to combine cryptographic protection with AI-based intrusion detection. Hybrid ECC-ANN frameworks demonstrate that confidentiality and anomaly detection can be integrated within the same MQTT security design, recognizing that encryption alone cannot prevent availability-oriented or behavioral attacks Karimunda et al. (2025). However, these approaches often lack formal verification, controlled key refresh, and inline broker admission control. Other cloud-oriented frameworks combine hybrid encryption with ensemble ML detection over encrypted MQTT traffic Qasem et al. (2026). While these works show that ML detection can remain effective when payloads are protected, their detection mechanisms are typically implemented as external cloud-processing pipelines rather than MQTT-aware inline proxy mechanisms.

Overall, the reviewed literature addresses MQTT security through partially separated directions. Vulnerability studies characterize the attack surface; TLS and lightweight-cryptography studies evaluate channel protection and overhead; formal methods analyze protocol-level security properties; and ML-based IDS studies focus on traffic classification. The representative studies reviewed in this section do not combine all of the following within one evaluated MQTT-aware workflow: formally analyzed session establishment, authenticated payload protection, controlled key refresh, metadata-only behavioral detection, and Proxy-side enforcement before Broker admission. MQTT-HS integrates these functions within a unified prototype while preserving payload confidentiality outside the trusted Proxy boundary.

3 Preliminaries and system model

This section presents the technical foundations and system model of the proposed MQTT-HS framework. It defines the cryptographic mechanisms, system entities, metadata model, adversary assumptions, protected assets, and main security objectives.

3.1 Cryptographic primitives

MQTT-HS uses Elliptic Curve Diffie–Hellman (ECDH) over the NIST P-256 curve to establish fresh shared secrets between MQTT clients and the Security Proxy. Each endpoint generates an ephemeral key pair and exchanges only the public key; the resulting shared secret is computed locally and is not transmitted.

Key confirmation during rekeying uses HMAC-SHA256 under the candidate session key. The confirmation input includes the transaction identifier and the current and proposed key epochs, binding the proof to the corresponding rekey transaction and epoch transition.

HKDF-SHA256 processes the raw ECDH output to derive a 32-byte symmetric session key using a session-specific salt and protocol context information. This derivation binds the resulting key to its authenticated session context and avoids using the raw ECDH secret directly.

After session establishment, MQTT payloads are protected using AES-256-GCM. Each encryption operation uses a fresh 96-bit nonce and produces an authentication tag for confidentiality and integrity. Envelopes that fail AES-GCM authentication are rejected. MQTT-HS uses PKI-issued X.509 certificates and ECDSA-SHA256 signatures to authenticate Publishers, Subscribers, and the Security Proxy. Certificates are validated against the configured Trust Store before session activation.

Each protected envelope authenticates its protocol and routing context as AES-GCM additional authenticated data (AAD). The AAD includes the protocol version, client and session identifiers, key epoch, topic identifier, authenticated message index, and security-relevant MQTT header fields. These fields remain visible for routing and metadata-based classification, while unauthorized modification causes AES-GCM authentication failure. Timing features are computed locally by the Proxy from authenticated envelope arrival events rather than accepted from sender-provided timestamps.

3.2 System entities and architecture

As illustrated in Fig. 1, MQTT-HS extends the standard MQTT publish/subscribe architecture by placing a Security Proxy between MQTT clients and the MQTT Broker. This placement preserves the Broker's standard routing role while enabling security processing and enforcement before Broker admission without modifying its implementation.

The main entities are the Publisher, Subscriber, Security Proxy, MQTT Broker, PKI/Trust Store, Cryptographic

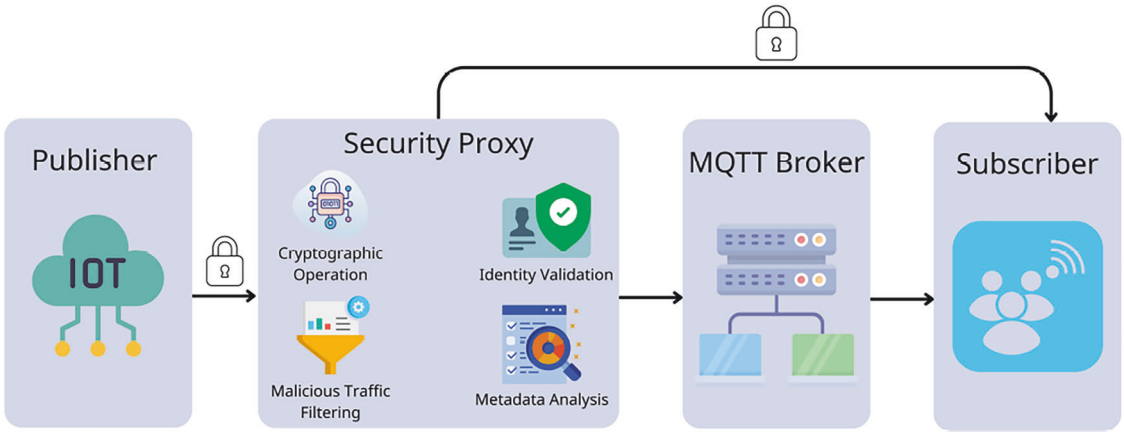


Fig. 1 High-level architecture of the proposed MQTT-HS framework

Engine, and hierarchical ML-IDS. The Publisher encrypts application payloads before transmission, while the Subscriber establishes an independent session with the Proxy and decrypts delivered messages. The Security Proxy performs mutual authentication, ECDH/HKDF session establishment, AES-256-GCM processing, metadata extraction, ML-IDS inference, enforcement, subscriber-specific re-encryption, and key refresh. The Broker receives validated and re-encrypted traffic for topic-based delivery.

Each Publisher and Subscriber maintains an independent secure session with the Security Proxy. The Publisher–Proxy session key protects Publisher traffic received by the Proxy, while each Subscriber–Proxy session key protects traffic delivered to the corresponding Subscriber. This separation prevents one Subscriber session key from decrypting traffic protected for another Subscriber and establishes the Proxy as the cryptographic and behavioral enforcement boundary.

3.3 Protocol-safe metadata model

The ML-IDS operates on protocol-safe structural metadata, enabling behavioral detection without exposing plaintext payload content to the classification pipeline. Its features include message length, locally measured inter-arrival time, MQTT control type, QoS level, connection flags, fixed-header flags, and transport-level indicators. Sender-provided fields used for classification or routing are authenticated as AES-GCM additional authenticated data, while timing features are computed locally by the Security Proxy. Metadata enters the session-specific window only after the envelope passes the session, epoch, replay, message-index, and AES-GCM checks. This ordering prevents invalid, replayed, or stale envelopes from affecting behavioral detection. Authenticated events are organized into windows of $W = 40$ messages with a stride of $S = 20$ messages to capture sequential traffic behavior. Authentication failures occurring before session establishment are handled separately through protocol-level enforcement.

3.4 Threat model

MQTT-HS considers a passive network observer and an active Dolev–Yao network adversary. The passive observer can inspect ciphertexts, visible protocol metadata, packet sizes, timing patterns, and communication relationships but cannot access endpoint or Proxy memory. Payload contents remain encrypted outside the trusted Proxy boundary, while packet sizes, timing patterns, and communication relationships remain observable.

The active adversary controls the communication channel and may intercept, modify, inject, delay, drop, duplicate, and replay messages exchanged among clients, the Security Proxy, and the MQTT Broker. The adversary may also attempt certificate-based impersonation, malformed-message injection, flooding, repeated authentication abuse, and SlowITe-style resource exhaustion. MQTT-HS applies certificate and signature verification, authenticated key establishment, AES-256-GCM protection, AAD validation, epoch and replay checks, controlled key refresh, and metadata-based enforcement against the corresponding authentication, integrity, replay, and behavioral threats.

The adversary does not compromise the physical hardware, private keys, active session keys, or protected runtime state of the IoT endpoints or Security Proxy. Hardware-level side-channel attacks are excluded from the threat model.

The Security Proxy is part of the MQTT-HS trusted computing base. It holds its authentication key, active Publisher–Proxy and Subscriber–Proxy session keys, replay and session state, authorization mappings, metadata windows, and transient plaintext required for subscriber-specific re-encryption. The threat model assumes the confidentiality and integrity of the Proxy execution environment, process memory, and key storage.

Compromise of the Security Proxy may expose active session keys, transient plaintext, routing state, and traffic

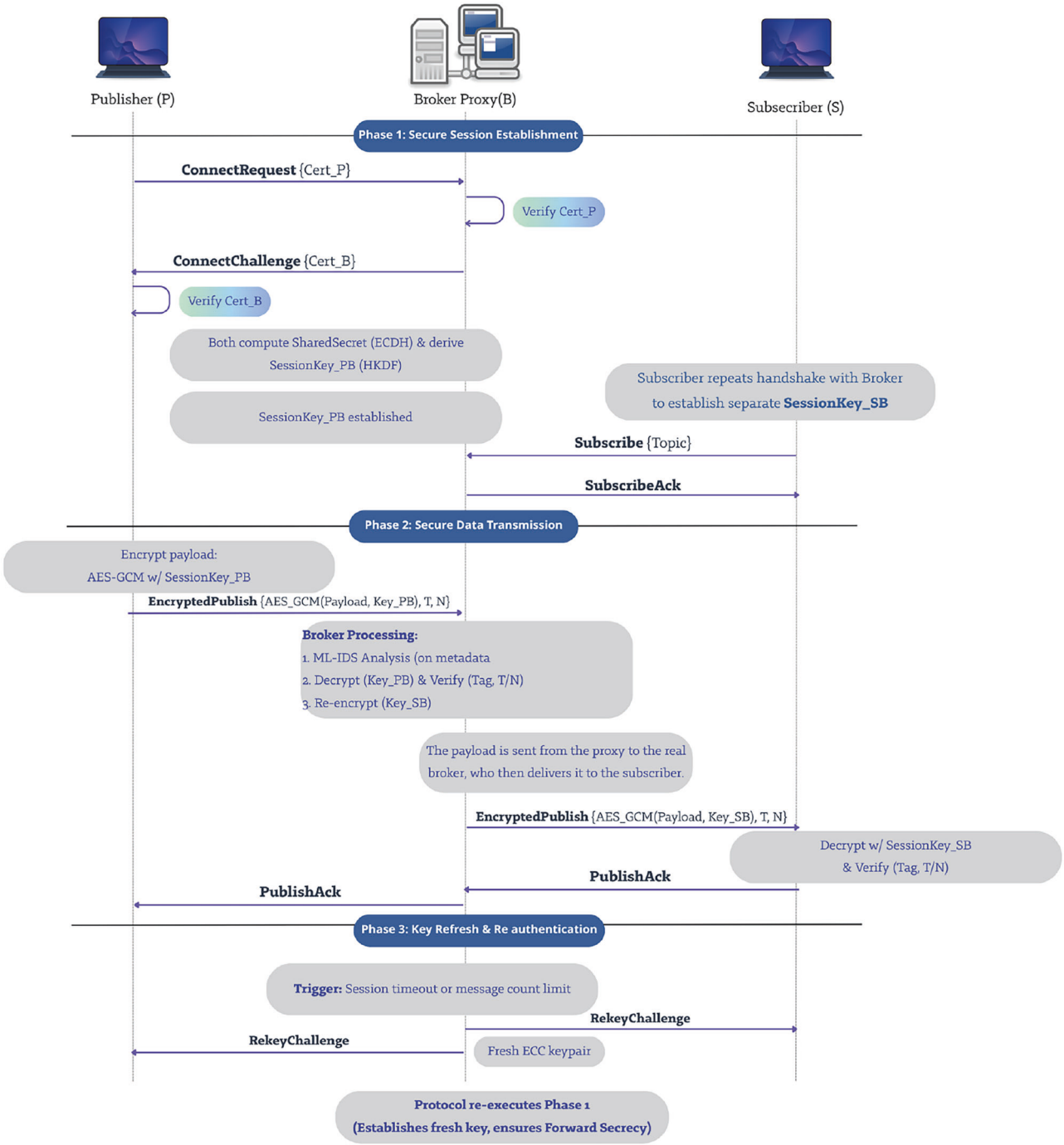


Fig. 2 Operational workflow of the MQTT-HS protocol

processed during the affected epoch. Protection of earlier epochs depends on erasing expired session keys and ephemeral private material after key derivation and epoch transition.

The primary network-adversary model excludes endpoint and Proxy compromise during protocol execution. Later key disclosure is considered only as a post-session security objective under the assumption that expired session keys and ephemeral private material have been securely erased.

3.5 Security assets and design goals

The protected assets are endpoint private keys, session keys, payload contents, authenticated protocol metadata, replay state, and authorization mappings. MQTT-HS targets

payload confidentiality and integrity, mutual certificate-based authentication, transcript-bound key establishment, replay resistance, authenticated visible metadata, forward secrecy through controlled session-key refresh, metadata-based behavioral detection, Subscriber-session isolation, and enforcement before Broker admission.

4 Proposed MQTT-HS protocol

This section presents MQTT-HS, a proxy-mediated security overlay for MQTT communication. MQTT-HS preserves the Broker's routing role while adding certificate-based mutual authentication, authenticated payload protection, metadata-based intrusion detection, Subscriber-specific cryptographic isolation, and controlled key refresh. The protocol operates through three main phases: secure session establishment, protected data transmission with Subscriber-specific routing, and threshold-based key refresh.

4.1 Protocol overview

MQTT-HS places a Security Proxy between MQTT clients and the MQTT Broker. The Proxy authenticates clients, establishes session keys, extracts protocol-safe metadata, applies hierarchical ML-based detection, and enforces protocol-level and behavioral decisions before Broker admission. The Broker retains its standard topic-based routing role without implementation changes.

Unlike end-to-end client-to-client encryption, MQTT-HS follows a proxy-mediated protection model. Each Publisher and Subscriber establishes an independent secure session with the Security Proxy. The Publisher encrypts each payload under its Publisher–Proxy session key. After authentication and behavioral processing, the Proxy re-encrypts the payload separately under each authorized Subscriber–Proxy session key before forwarding it through the Broker. Payload contents therefore remain encrypted from external observers and the Broker, while the trusted Proxy performs decryption, enforcement, and Subscriber-specific re-encryption.

Publishers are not required to know the identities or number of Subscribers associated with a topic. The Proxy maintains an Authorized Subscriber Session Table that maps protected topics to authenticated Subscriber–Proxy sessions. The table is populated during authenticated subscription establishment and supports authorization and Subscriber-specific cryptographic fan-out. Figure 2 illustrates session establishment, protected-message processing, ML-IDS enforcement, Broker forwarding, and key refresh.

4.2 Secure session establishment

The first phase establishes a mutually authenticated session between a Publisher or Subscriber and the Security Proxy. The client generates an ephemeral ECDH key pair and a fresh nonce n_C, then sends a signed `ConnectRequest` containing its X.509 certificate, ephemeral public key, requested role, nonce, and protocol version. The ECDSA-SHA256 signature binds the authenticated client identity to the request context.

Before deriving session-key material, the Proxy validates the certificate chain, validity period, Trust Store anchor, configured client identity and role, and request signature. Validation failure terminates the handshake.

After successful validation, the Proxy generates an ephemeral ECDH key pair and fresh nonce n_P, then returns a signed `ConnectChallenge` containing its certificate, ephemeral public key, nonce, and accepted session context. The client validates the Proxy certificate and challenge signature before deriving the session key.

Let (sk_C, pk_C) and (sk_P, pk_P) denote the ephemeral ECDH key pairs of the client and Proxy, respectively. Both endpoints compute the same shared secret:

$$SS = \mathrm{ECDH}(sk_C, pk_P) = \mathrm{ECDH}(sk_P, pk_C). \quad (1)$$

The *salt* is derived from the exchanged nonces, while *info* binds the key to the authenticated session context, including the protocol version, endpoint identities, negotiated role, ephemeral public keys, and nonces. Session state is activated after certificate and signature validation and successful key derivation. Algorithm 1 summarizes the procedure.

Algorithm 1 Authenticated handshake and session-key derivation

Require: Client C, Security Proxy P, PKI Trust Store
Ensure: Activated session state or handshake failure
1: C generates ephemeral key pair (*cpriv*, *cpub*)
2: C generates fresh nonce n_C
3: $ctx_C \Leftarrow$ (*ClientID*, *Role*, *cpub*, n_C, *ProtocolVersion*)
4: $sig_C \Leftarrow \mathrm{ECDSA-Sign}(\mathrm{sk}_C^{\mathrm{auth}}, \mathrm{ctx}_C)$
5: $C \rightarrow P$: `ConnectRequest`{$Cert_C$, ctx_C, sig_C}
6: P validates $Cert_C$, *ClientID*, *Role*, and sig_C
7: **if** certificate, identity, role, or signature validation fails **then**
8: `LogSecurityEvent("invalid_connect_request")`
9: **return** HandshakeFailure
10: **end if**
11: P generates ephemeral key pair (*ppriv*, *ppub*)
12: P generates fresh nonce n_P
13: $ctx_P \Leftarrow$ (ctx_C, *ProxyID*, *ppub*, n_P)
14: $sig_P \Leftarrow \mathrm{ECDSA-Sign}(\mathrm{sk}_P^{\mathrm{auth}}, \mathrm{ctx}_P)$

15: $P \rightarrow C$: `ConnectChallenge`{$Cert_P$, $ProxyID$, $ppub$, n_P, sig_P}
16: C validates $Cert_P$, $ProxyID$, and sig_P
17: **if** Proxy certificate or signature validation fails **then**
18: `LogSecurityEvent ("invalid_proxy_challenge")`
19: **return** HandshakeFailure
20: **end if**
21: $SS_C \Leftarrow \text{ECDH}(cpriv, ppub)$
22: $SS_P \Leftarrow \text{ECDH}(ppriv, cpub)$
23: $salt \Leftarrow H(n_C \| n_P)$
24: $info \Leftarrow H(ctx_C \| ProxyID \| ppub \| n_P)$
25: C derives $K_C \leftarrow \text{HKDF-SHA256}(SS_C, salt, info)$
26: P derives $K_P \leftarrow \text{HKDF-SHA256}(SS_P, salt, info)$
27: *Pinitializes* {*ClientID*, *Role*, K_{session}, *CurrentEpoch*, *MsgCounter*, *SessionTimer*, *ReplayState*, *SlidingWindow*}
28: Erase *cpriv*, *ppriv*, and temporary shared-secret material after successful derivation
29: **return** K_{session}

4.3 Secure data transmission and subscriber-specific routing

After session establishment, MQTT-HS protects each MQTT payload using AES-256-GCM. The visible protocol and routing context is bound to the protected message as additional authenticated data (AAD). The AAD includes the protocol version, client and session identifiers, key epoch, topic identifier, authenticated message index, and security-relevant MQTT header fields.

For plaintext payload M, nonce N, and authenticated metadata AAD, the client computes:

$$(C, T) = \text{AES} - 256 - \text{GCM.Enc}(K_{\text{session}}, N, M, AAD), \tag{2}$$

where C is the ciphertext and T is the authentication tag. The protected envelope carries N, C, T, the key epoch, and the visible AAD fields.

Upon receiving a protected envelope, the Security Proxy first verifies the Publisher session, key epoch, nonce freshness, AES-GCM authentication tag, and authenticated message index. Only after these checks succeed does the Proxy admit the associated metadata event to the session-specific ML sliding window. This ordering prevents unauthenticated, modified, replayed, or stale envelopes from entering the behavioral feature window.

The behavioral state remains `Pending` until the session window contains $W = 40$ authenticated events. Protocol-level validation remains active during warm-up, and accepted messages continue through the protected forwarding path. Subsequent windows are evaluated with a stride of $S = 20$ events.

Stage 0 and Stage 1 use the Direct-11 feature representation. If Stage 1 predicts the Denial-Family, Stage 2A receives the specialized Pruned-10 representation. A malicious decision causes the current envelope to be blocked or throttled according to the enforcement policy. A legitimate or Pending decision allows processing to continue.

For an allowed message, the Proxy retrieves the authorized Subscribers for the authenticated topic and produces a separate protected envelope using each Subscriber–Proxy session key. The Publisher therefore encrypts and sends each application message only once, while subscriber-specific cryptographic fan-out is performed by the Security Proxy. Compromise of one Subscriber–Proxy key does not expose messages protected under the keys of other Subscribers. Algorithm 2 summarizes this procedure.

Here, `ForwardedPending` indicates that the envelope passed the protocol-level checks and was forwarded while the behavioral window remained incomplete.

Algorithm 2 Authenticated message processing and subscriber-specific routing

Require: Protected envelope E_{in} = {*ClientID*, *Epoch*, *Nonce*, CT_{in}, *AAD*}, window size $W = 40$, stride $S = 20$
Ensure: Forwarded, ForwardedPending, or Blocked
1: *session* $\Leftarrow$ `GetPublisherSession`(E_{in}. *ClientID*)
2: **if** *session* $= \varnothing$ **or** *session*. *Role* $\neq$ `"Publisher"` **then**
3: `DropAndLog`(E_{in},`"invalid_publisher_session"`)
4: **return** Blocked
5: **end if**
6: **if** E_{in}. *Epoch* $\neq$ *session*. *CurrentEpoch* **then**
7: `DropAndLog`(E_{in},`"epoch_mismatch"`)
8: **return** Blocked
9: **end if**
10: **if** E_{in}. *Nonce* $\in$ *session*. *AcceptedNonces* **then**
11: `DropAndLog`(E_{in},`"duplicate_nonce"`)
12: **return** Blocked
13: **end if**
14: (*status*, *plaintext*) $\Leftarrow$ `AESGCMDecryptAndVerify` ($K_{Publisher\text{-}Proxy}$, E_{in}. *Nonce*, E_{in}. CT_{in}, E_{in}. *AAD*)
15: **if** *status* $=$ `Failure` **then**
16: `DropAndLog`(E_{in},`"authentication_failure"`)
17: **return** Blocked
18: **end if**
19: *msgIndex* $\Leftarrow$ `GetAuthenticatedMessageIndex`(E_{in}. *AAD*)
20: **if** *msgIndex*$\leq$*session*. *LastMessageIndex* **then**
21: `DropAndLog`(E_{in},`"non_increasing_message_index"`)

22: **return** Blocked
23: **end if**
24: `CommitReplayState`(*session*, E_{in}. *Nonce*, *msgIndex*)
25: *metadata* ⇐ `ExtractAuthenticatedMetadata`(E_{in}. *AAD*)
26: *metadata*. *IAT* ⇐ `ComputeLocalInterArrivalTime` (*session*)
27: *decision* ⇐ `HierarchicalMLIDS`(*metadata*, *session*. *SlidingWindow*, *W*, *S*)
28: **if** *decision* is malicious **then**
29: `LogSecurityEvent`(*metadata*, *decision*)
30: **return** Blocked
31: **end if**
32: *subscriberSet* ⇐ `GetAuthorizedSubscribers` (E_{in}. *AAD*. *topic*)
33: **for each** subscriber in *subscriberSet* **do**
34: *subSession* ⇐ `GetSubscriberSession` (*subscriber*, E_{in}. *AAD*. *topic*)
35: AAD_{out} ⇐ `BuildSubscriberAAD`(E_{in}. *AAD*, *subSession*)
36: E_{out} ⇐ `ProtectForSubscriber` (*subSession*. *Key*, *plaintext*, AAD_{out})
37: `ForwardToBroker` (E_{out}, AAD_{out}. *topic*, *subscriber*)
38: **end for**
39: `Increment`(*session*. *MsgCounter*)
40: **if** *decision* = `Pending` **then**
41: **return** `ForwardedPending`
42: **else**
43: **return** `Forwarded`
44: **end if**

4.4 Key refresh and forward secrecy

MQTT-HS applies Proxy-managed key refresh when either the session age or protected-message count reaches its configured threshold. The evaluated implementation used thresholds of 300 s and 40 protected messages.

When rekeying is triggered, the Proxy creates a unique rekey transaction identifier, records the current and proposed key epochs, and generates fresh ephemeral ECDH material. It sends a `RekeyChallenge` containing the transaction identifier, epoch values, and Proxy ephemeral public key. The client validates the transaction state and generates fresh ephemeral ECDH material for the same transaction.

Both endpoints derive a candidate key using fresh ECDH material and HKDF-SHA256. The client additionally generates an HMAC-SHA256 proof under the candidate key. This proof binds the candidate key to the transaction identifier and current and proposed epochs, allowing the Proxy to verify that both endpoints derived the same next-epoch key without transmitting that key.

Let $(sk_{C,i+1}, pk_{C,i+1})$ and $(sk_{P,i+1}, pk_{P,i+1})$ denote the fresh client and Proxy key pairs. Both endpoints derive the candidate shared secret:

$$SS_{i+1} = \mathrm{ECDH}(sk_{C,i+1}, pk_{P,i+1}) = \mathrm{ECDH}(sk_{P,i+1}, pk_{C,i+1}), \quad (3)$$

and derive the next-epoch key as:

$$K_{i+1} = \mathrm{HKDF} - \mathrm{SHA256}(SS_{i+1}, salt_{i+1}, info_{i+1}). \quad (4)$$

The current key remains active while the candidate key is negotiated. The candidate key is activated after the authenticated acknowledgement and commit sequence, and the previous key is erased after a bounded transition interval. Because each epoch key is derived from fresh ephemeral ECDH material rather than from the preceding session key, disclosure of a later epoch key does not provide the cryptographic inputs required to reconstruct an earlier epoch key. This forward-secrecy property depends on erasing expired session keys, ephemeral private values, and temporary shared-secret material after the bounded transition interval.

4.5 Rekey error handling and packet-loss recovery

MQTT-HS treats rekeying as an authenticated transaction identified by the client identifier, current epoch, proposed epoch, and a unique transaction identifier. Lost control messages trigger bounded retransmission using the same transaction identifier. Duplicate challenges, responses, acknowledgements, and commits are processed idempotently and do not independently advance the key epoch.

The `RekeyChallenge`, `RekeyResponse`, and `RekeyConfirm` are authenticated under the current session key. The response additionally carries an HMAC-SHA256 proof computed under the candidate key. The client authenticates `RekeyAck` under the candidate key, and the Proxy authenticates `RekeyCommit` under the same key, providing bidirectional confirmation of the next-epoch key.

When the client receives a duplicate challenge for a pending transaction, it reuses the cached ephemeral public key, candidate key, and response proof associated with that transaction instead of generating independent key material. Similarly, the Proxy treats duplicate responses and acknowledgements as references to the existing transaction.

The client stores the candidate key as pending while retaining the current active key. It activates the pending key after receiving a valid `RekeyCommit` or a valid authenticated envelope carrying the proposed next epoch. Separate replay state is maintained for the current and pending epochs during the bounded transition interval.

If the retry limit is exhausted, the candidate keys and pending transaction state are erased. The current session

continues only if the current key remains within the permitted lifetime and volume policy. Otherwise, the Proxy terminates the session and requires a new authenticated handshake.

Algorithm 3 summarizes the controlled key-refresh and recovery procedure.

4.6 Proxy control logic

The Security Proxy processes handshake requests, subscription updates, protected envelopes, rekey control messages, and timer events. Rekey timers are handled independently of application-message arrival, allowing pending transactions to be retransmitted or expired according to Section 4.5. Algorithm 4 summarizes the event loop.

Algorithm 3 Controlled key refresh with packet-loss recovery

Require: Active session, ExpiryThreshold, VolumeThreshold, RekeyTimeout, MaxRetries

Ensure: Refreshed key, preserved current key, or terminated session

if *SessionTimer* < *ExpiryThreshold* **and** *MsgCounter* < *VolumeThreshold* **then**
  return K_{current}
end if
currentEpoch ⇐ *Session*. *CurrentEpoch*
nextEpoch ⇐ *currentEpoch* + 1
txID ⇐ `GenerateTransactionID()`
P generates ($ppriv_{new}$, $ppub_{new}$)
retryCount ⇐ 0
while *retryCount* < *MaxRetries* **do**
  P → *C*: `RekeyChallenge{`*txID*, *currentEpoch*, *nextEpoch*, $ppub_{new}$`}`
  Authenticate the challenge under K_{current}
  if no valid challenge is received before *RekeyTimeout* **then**
    retryCount ⇐ *retryCount* + 1
    continue
end if
if *C* has no pending state for *txID* **then**
    C generates ($cpriv_{new}$, $cpub_{new}$)
  $K_{C,candidate}$ ⇐ `DeriveRekeyKey(`$cpriv_{new}$, $ppub_{new}$, *txID*, *nextEpoch*`)`
    $proof_C$ ⇐ HMAC-SHA256($K_{C,candidate}$, *txID*‖*currentEpoch*‖*nextEpoch*)
    Cache ($cpub_{new}$, $K_{C,candidate}$, $proof_C$) under *txID*
else
    Reuse cached response state for *txID*
end if
C → *P*: `RekeyResponse{`*txID*, *currentEpoch*, *nextEpoch*, $cpub_{new}$, $proof_C$`}`
  Authenticate the response under K_{current}
  if no valid response is received before *RekeyTimeout* **then**
    retryCount ⇐ *retryCount* + 1
    continue
  end if
$K_{P,candidate}$ ⇐ `DeriveRekeyKey(`$ppriv_{new}$, $cpub_{new}$, *txID*, *nextEpoch*`)`
if `VerifyHMAC(`$K_{P,candidate}$, $proof_C$, *txID*‖*currentEpoch*‖*nextEpoch*`)=False` **then**
    `DiscardInvalidResponse(`*txID*`)`
    retryCount ⇐ *retryCount* + 1
    continue
  end if
  P → *C*: `RekeyConfirm{`*txID*, *nextEpoch*`}`
  Authenticate `RekeyConfirm` under K_{current}
  C stores $K_{C,candidate}$ as pending
  C → *P*: `RekeyAck{`*txID*, *nextEpoch*`}`
  Authenticate `RekeyAck` under $K_{C,candidate}$
  if a valid acknowledgement is received **then**
    P activates $K_{P,candidate}$ for *nextEpoch*
    Initialize separate Proxy replay state for *nextEpoch*
    P → *C*: `RekeyCommit{`*txID*, *nextEpoch*`}`
    Authenticate `RekeyCommit` under $K_{P,candidate}$
    C activates the pending key after a valid `RekeyCommit` or a valid next-epoch envelope
    Session. *CurrentEpoch* ⇐ *nextEpoch*
    Reset(*SessionTimer*) and Reset(*MsgCounter*)
    Retain K_{current} for the bounded transition interval
    Erase the previous key and ephemeral material after the transition interval expires
    return $K_{P,candidate}$
    end if
    retryCount ⇐ *retryCount* + 1
end while
Erase candidate keys and pending transaction state
if the current key remains within the permitted policy **then**
    return K_{current}
else
    `TerminateSession()`
    return `HandshakeRequired`
end if

5 Privacy-preserving hierarchical machine learning-IDS

This section presents the hierarchical machine learning-based intrusion detection system (ML-IDS) integrated into the Security Proxy. The ML-IDS complements the cryptographic and protocol-level controls in Section 4 by classifying authenticated protocol-safe metadata and locally

Algorithm 4 Security Proxy main control loop

Require: PKI Trust Store, trained ML-IDS hierarchy, rekey policy, enforcement policy

Ensure: Event processing according to the MQTT-HS control policy

```
LoadPKITrustStore()
LoadXGBoostHierarchy (Stage0, Stage1, Stage2A)
InitializeAuthorizedSubscriberSessionTable()
InitializeSessionAndTimerState()
while Security Proxy is active do
    event ⇐ ReceiveNetworkOrTimerEvent()
    if event. type=HANDSHAKE_REQUEST then
    AuthenticatedHandshakeAndSessionKeyDerivation (event. client, Proxy)
    else if event. type=SUBSCRIPTION_UPDATE then
ValidateSubscriptionAuthorization (event. client, event. topic)
    UpdateAuthorizedSubscriberSessionTable (event. client, event. topic)
    else if event. type = PROTECTED_MQTT_ENVELOPE then
    result ⇐ AuthenticatedMessageProcessingAndRouting (event. envelope, Proxy)
    if result ≠ Blocked then
    EvaluateRekeyPolicy (event. client, ExpiryThreshold, VolumeThreshold)
    end if
  else if event. type = REKEY_CONTROL_MESSAGE then
ProcessAuthenticatedRekeyMessage (event. client, event. message)
  else if event. type = REKEY_TIMER_EVENT then
 ProcessPendingRekeyTimeout (event. client, RekeyTimeout, MaxRetries)
  else
LogSecurityEvent (event, "unsupported_or_invalid_event")
    Drop(event)
  end if
end while
```

derived timing features without using plaintext payload content.

Figure 3 summarizes the offline training and online inference pipelines. Offline processing cleans the raw MQTT data, constructs protocol-safe features and chronological sliding windows, and trains the hierarchical classifiers. During online operation, authenticated metadata events enter session-specific windows after session, epoch, replay, message-index, and AES-GCM authentication checks succeed. Complete windows are then classified before the Proxy applies the resulting enforcement decision.

5.1 Dataset preparation and cleaning

The ML-IDS was developed using MQTTEEB-D, an MQTT-specific intrusion-detection dataset containing benign telemetry and multiple attack categories, including Flood, Bruteforce, Malformed, DoS, and SlowITe traffic Aqachtoul et al. (2025).

The study uses the raw, unaggregated loop files to preserve event order, construct chronological sliding windows, and restrict the feature pipeline to information available during online operation.

To reduce temporal leakage, the dataset was divided at the loop level. Loop 1, Loop 3, and Loop 4 were used for training, while Loop 2 was isolated as the holdout evaluation loop. No window from Loop 2 was used to fit the preprocessing transformations or train the classifiers.

Rows with missing class labels were removed, and mixed-format MQTT and transport fields were standardized into numerical values. A unified MQTT control-type feature was derived from bits 7–4 of the MQTT Fixed Header to represent control-packet semantics without using payload content.

The original timing field was replaced with inter-arrival time recomputed from consecutive events within each loop. This construction aligns offline feature generation with online operation, where the Security Proxy computes inter-arrival time from accepted envelope events. Figure 4 shows the resulting timing distributions across the traffic classes, including the contrasting rate characteristics of Flood and SlowITe traffic.

5.2 Protocol-safe feature engineering

The ML-IDS uses structural metadata that is visible or locally derivable during protected MQTT communication. Payload bytes, application values, and features exposing payload semantics are excluded.

Features were retained when they were available during protected runtime operation, relevant to MQTT traffic behavior, and independent of plaintext payload content.

Runtime features combine authenticated envelope fields with inter-arrival time computed locally by the Security Proxy. Only events accepted by the protected-message processing pipeline in Section 4.3 enter the behavioral window.

The final runtime-safe feature schemas are summarized in Table 2. Stage 0 and Stage 1 use the 11-feature Direct-11 representation, while Stage 2A uses the specialized 10-feature Pruned-10 representation for DoS and SlowITe discrimination.

5.3 Sliding-window sequence construction

Flood, brute-force, DoS, and SlowITe behavior may depend on timing, repetition, rate, and control-message sequences rather

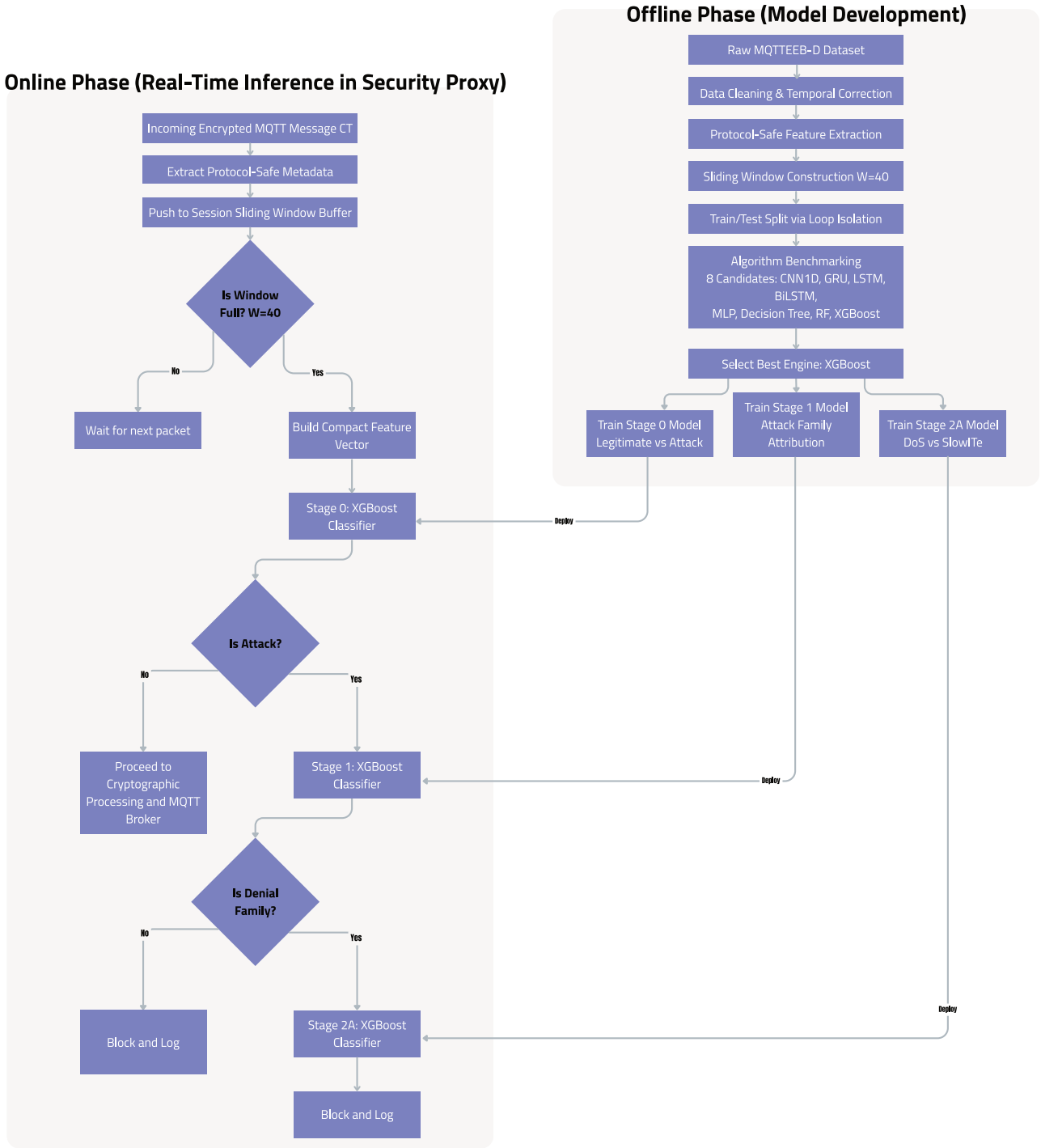


Fig. 3 Workflow of the metadata-based hierarchical ML-IDS integrated into MQTT-HS

than individual packets. The cleaned metadata events were therefore transformed into chronological sliding windows.

Each window contains $W = 40$ consecutive events with a stride of $S = 20$. The primary supervised pipeline used a purity threshold of 1.0, retaining a window only when all 40 events shared the same class label. This avoided assigning a single training label to mixed-class or legitimate-to-attack transition windows.

Window construction generated 11,122 candidate windows. Of these, 9574 pure windows were retained for the primary modeling pipeline, while 1548 mixed-label windows were excluded from primary training. Table 3

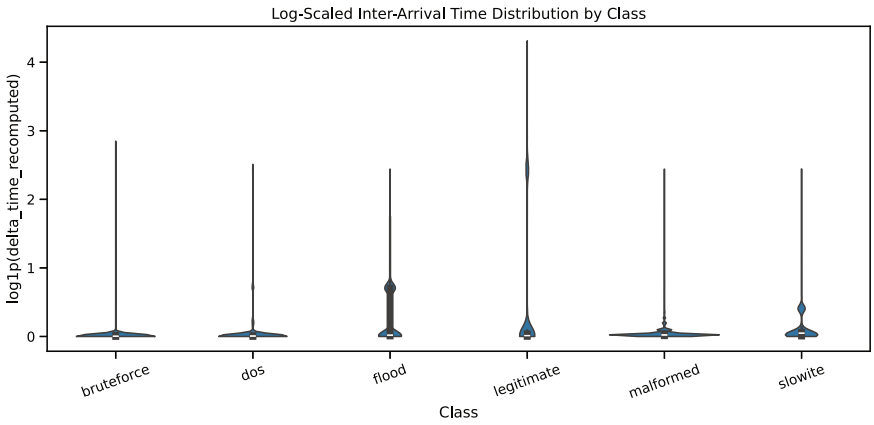


Fig. 4 Log-scaled distribution of recomputed inter-arrival time by traffic class

Table 2 Runtime-safe feature schemas used by the hierarchical ML-IDS

Stage / Representation	Feature Count	Feature List
Stage 0 and Stage 1: Direct-11	11	inter_arrival_time, frame_or_message_length, mqtt_control_type, qos_level, dup_flag, keepalive_value, connect_flags, connack_flags, clean_session_flag, header_flags, transport_header_signal
Stage 2A: Pruned-10	10	inter_arrival_time, frame_or_message_length, mqtt_control_type, qos_level, dup_flag, keepalive_value, connect_flags, clean_session_flag, transport_header_signal, control_type_switch_rate

summarizes the retained class distribution, and Fig. 5 presents its loop-wise distribution.

The exclusion of mixed windows applies only to primary supervised training. During runtime, the Security Proxy does not know the ground-truth composition of an incoming window and evaluates every complete authenticated window. Mixed-window behavior is examined separately through the post-hoc transition analysis reported in Section 7.10.4.

5.4 Hierarchical detection architecture and training

The ML-IDS uses a three-stage hierarchy that separates attack detection from attack-family attribution and DoS–SlowITe discrimination.

Stage 0 classifies each complete window as Legitimate or Attack. Windows classified as Legitimate do not proceed to later stages. Stage 1 assigns Attack windows to Bruteforce, Flood, Malformed, or Denial-Family. When Stage 1 predicts Denial-Family, Stage 2A distinguishes DoS from SlowITe using the Pruned-10 representation. Table 4 summarizes the stage labels.

The evaluated candidate classifiers included Decision Tree, Random Forest, XGBoost, CNN1D, GRU, LSTM, BiLSTM, and a flattened multilayer perceptron. Preprocessing parameters were fitted using only the training loops and then applied unchanged to the isolated holdout loop. Class imbalance was addressed using the weighting configuration associated with each training task.

Model selection considered balanced accuracy, macro F1-score, class-wise performance, inference latency, throughput, and serialized model size. XGBoost was selected based on the stage-wise predictive and runtime results reported in Section 7.10. The Proxy loads each model with the preprocessing artifacts required by its feature representation.

5.5 Runtime operation inside the Security Proxy

The online ML-IDS receives accepted metadata events from the protected-message processing pipeline in Section 4.3. The Proxy extracts authenticated structural fields, computes local inter-arrival time, and appends each event to its session-specific sliding window.

The behavioral state remains `Pending`, rather than Legitimate, until the buffer contains $W = 40$ events. Protocol-level controls remain active during warm-up, and accepted messages continue through Subscriber-specific protection and routing.

For each complete window, Stage 0 evaluates the Direct-11 representation. Attack predictions proceed to Stage 1, and Denial-Family predictions proceed to Stage 2A using the Pruned-10 representation. Legitimate predictions terminate the hierarchy at Stage 0.

A malicious decision is returned to the Security Proxy enforcement logic, which may block or throttle the current traffic according to the configured policy and records the client identifier, timestamp, authenticated metadata

Table 3 Final sliding-window distribution of MQTTEEB-D traffic categories

Class Label	Window Count	Description
Legitimate	512	Standard IoT telemetry and benign MQTT operations
Malformed	5033	Protocol-format anomalies and corrupted MQTT structures
DoS	1840	Standard denial-of-service attempts
Bruteforce	1136	Repeated MQTT connection or authentication attempts
SlowITe	746	Low-rate stealthy denial-of-service behavior
Flood	307	High-rate uniform message transmission
Total	**9574**	Sequence-level sliding-window dataset

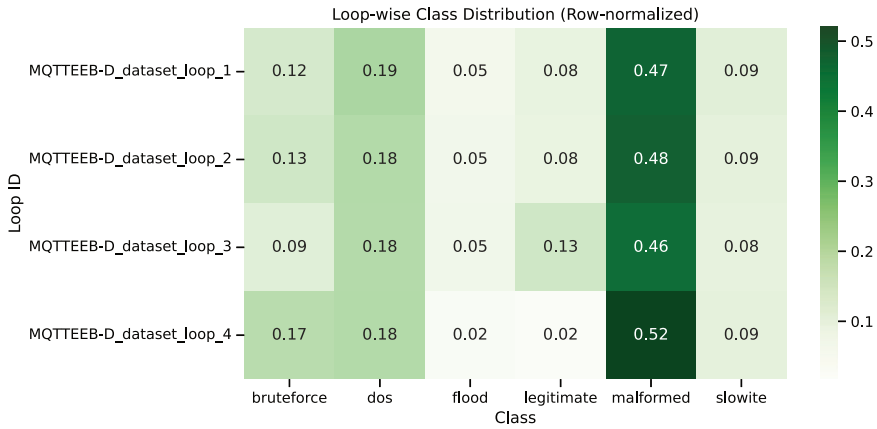


Fig. 5 Row-normalized loop-wise class distribution of the sliding windows

Table 4 Label encoding scheme for the hierarchical ML-IDS

Stage	Classification Task	Encoded Labels
Stage 0	Binary classification	0 = Legitimate, 1 = Attack
Stage 1	Attack-family attribution	0 = Bruteforce, 1 = Flood, 2 = Malformed, 3 = Denial-Family
Stage 2A	Denial-family discrimination	0 = DoS, 1 = SlowITe

summary, and predicted label. Plaintext payload content is not provided to the classifier or written to the ML-IDS event log.

The metadata-only restriction applies to ML classification; the trusted Proxy processes plaintext transiently for Subscriber-specific re-encryption but does not use it as an ML feature. Algorithm 5 summarizes hierarchical inference.

6 Security analysis and formal verification

This section analyzes MQTT-HS through an informal security argument and a ProVerif model of its cryptographic core. The analysis covers secrecy, authentication, replay resistance, message-flow agreement, and epoch-based key refresh under an active Dolev–Yao adversary.

Algorithm 5 Hierarchical metadata-based intrusion detection

Require: Authenticated metadata event m, session sliding window SW, window size $W = 40$, stride $S = 20$

Ensure: Detection status: Legitimate, Malicious(*label*), or Pending

```
Push(SW, m)
if Length(SW) < W then
    LogDetectionState ("Pending",Length (SW),W)
    return Pending
end if
features_D ⇐ BuildDirect11FeatureVector (SW)
stage0Decision ⇐ PredictStage0XGBoost (features_D)
if stage0Decision="Legitimate" then
    Slide(SW, S)
    return Legitimate
end if
stage1Decision ⇐ PredictStage1XGBoost (features_D)
if stage1Decision="Denial-Family" then
    features_P ⇐ BuildPruned10FeatureVector (SW)
    finalLabel ⇐ PredictStage2AXGBoost (features_P)
else
    finalLabel ⇐ stage1Decision
```

18: **end if**
19: `LogBehavioralDecision` (*m*, *finalLabel*)
20: `Slide`(*SW*, *S*)
21: **return** `Malicious`(*finalLabel*)

6.1 Informal security analysis

MQTT payloads are encrypted using AES-256-GCM before transmission. External observers and the MQTT Broker receive ciphertext rather than plaintext, while the trusted Security Proxy decrypts payloads transiently for enforcement and Subscriber-specific re-encryption. The ML-IDS uses authenticated structural metadata and does not use plaintext payload content.

AES-256-GCM authenticates the ciphertext and associated protocol context; modification of the ciphertext, nonce, authentication tag, or AAD causes authentication failure. Mutual authentication uses X.509 certificate validation and ECDSA-SHA256 transcript signatures. Certificates or signatures that fail the configured trust, validity, identity, or signature checks terminate session establishment.

Replay resistance uses key-epoch validation, per-session nonce tracking, and authenticated message indices. Envelopes carrying a stale epoch or previously accepted nonce are rejected before decryption, while non-increasing authenticated message indices are rejected after AES-GCM verification. Rejected envelopes do not enter the ML sliding window, increment the protected-message counter, trigger rekey evaluation, or reach the forwarding path.

MQTT-HS supports forward secrecy through fresh ephemeral ECDH material during initial session establishment and every key-refresh transaction. Each epoch key is derived independently using a fresh ECDH shared secret, salt, and protocol context rather than from the preceding session key. Under the stated erasure assumption, later compromise of a long-term authentication key or active epoch key does not disclose the ephemeral material required to reconstruct an expired epoch key.

Metadata-based detection evaluates Flood, SlowITe, and Malformed behavior after protected-session validation. Certificate and signature failures are handled before session-key derivation, and repeated signature failures activate temporary authentication blocking. Subscriber-specific re-encryption isolates delivery keys, so a compromised Subscriber–Proxy key does not decrypt envelopes protected for other Subscriber sessions.

6.2 ProVerif formal verification model

The cryptographic core of MQTT-HS was modeled in ProVerif under a symbolic Dolev–Yao adversary that controls the public channel but cannot break the modeled cryptographic primitives.

The model covers client–Proxy session establishment, symbolic ECDH-style secrets, epoch-key derivation, authenticated encryption, and key refresh from epoch 0 to epoch 1. Publisher-side and Subscriber-side protected flows are represented through authentication and message-flow correspondence events. ML classification and behavioral attack detection are evaluated empirically in Section 7.

The first property group covers base-secret, epoch-key, and protected-payload secrecy. Representative queries include:

```
query attacker(basePB).
query attacker(baseBS).
query attacker(kdf_epoch(basePB, sidPB,
epoch0)).
query attacker(kdf_epoch(basePB, sidPB,
epoch1)).
query attacker(kdf_epoch(baseBS, sidBS,
epoch0)).
query attacker(kdf_epoch(baseBS, sidBS,
epoch1)).
```

Here, `basePB` and `baseBS` represent the abstract base secrets of the Publisher-side and Subscriber-side channels, while `epoch0` and `epoch1` represent the initial and refreshed key epochs.

Second, mutual authentication and replay protection are modeled using injective agreement events. For example:

```
query h-2ptinj-event(pub_phase1_complete
(sidPB)) ==> inj-event(secproxy_pub_accept
(sidPB)).
```

This injective correspondence requires each Publisher-side completion event to match a distinct Proxy acceptance event for the same modeled session identifier. Corresponding queries cover Subscriber-side completion, rekey completion, and protected-message flow.

Third, the model represents key refresh through an epoch transition from `epoch0` to `epoch1`. Rekey events, such as:

```
event secproxy_rekey_complete(sidPB,
epoch1).
```

mark authenticated completion of the transition to refreshed key material. Separate secrecy queries evaluate the initial and refreshed epoch keys and their protected payloads. These queries establish epoch-key secrecy across the modeled key-refresh execution. The concrete forward-secrecy property additionally relies on fresh ephemeral

```
--------------------------------------------------------------
Verification summary:

Non-interference payload0 is true.

Non-interference payload1 is true.

Query not attacker(basePB[]) is true.

Query not attacker(baseBS[]) is true.

Query not attacker(payload0[]) is true.

Query not attacker(payload1[]) is true.

Query not attacker(kdf_epoch(basePB[],sidPB[],epoch0[])) is true.

Query not attacker(kdf_epoch(basePB[],sidPB[],epoch1[])) is true.

Query not attacker(kdf_epoch(baseBS[],sidBS[],epoch0[])) is true.

Query not attacker(kdf_epoch(baseBS[],sidBS[],epoch1[])) is true.

Query inj-event(pub_phase1_complete(sidPB[])) ==> inj-event(secproxy_pub_accept(sidPB[])) is true.

Query inj-event(sub_phase1_complete(sidBS[])) ==> inj-event(secproxy_sub_accept(sidBS[])) is true.

Query inj-event(secproxy_pub_rekey_complete(sidPB[],epoch1[])) ==> inj-event(pub_rekey_accept(sidPB[],epoch1[])) is true.

Query inj-event(secproxy_sub_rekey_complete(sidBS[],epoch1[])) ==> inj-event(sub_rekey_accept(sidBS[],epoch1[])) is true.

Query inj-event(secproxy_receive_epoch0(sidPB[])) ==> inj-event(pub_send_epoch0(sidPB[])) is true.

Query inj-event(secproxy_receive_epoch1(sidPB[])) ==> inj-event(pub_send_epoch1(sidPB[])) is true.

Query inj-event(sub_receive_epoch0(sidBS[])) ==> inj-event(secproxy_forward_epoch0(sidBS[])) is true.

Query inj-event(sub_receive_epoch1(sidBS[])) ==> inj-event(secproxy_forward_epoch1(sidBS[])) is true.

--------------------------------------------------------------
```

Fig. 6 ProVerif verification summary for MQTT-HS secrecy, authentication, rekey, and message-flow queries

ECDH generation and the erasure of expired session keys and ephemeral private material.

6.3 Formal verification results

ProVerif verified all secrecy, non-interference, authentication, rekey, and message-flow correspondence queries included in the symbolic model. As shown in Fig. 6, the modeled base secrets, epoch keys, and protected payloads remained secret. The correspondence results also verified authenticated progression across session establishment, key refresh, forwarding, and delivery. These symbolic results complement the forward-secrecy design provided by fresh ephemeral ECDH derivations and secure erasure in the concrete protocol.

The verified property groups and their outcomes are summarized in Table 5.

The verified queries cover base-secret and epoch-key secrecy, protected-payload confidentiality, non-interference, injective session and rekey agreement, and authenticated message-flow correspondences for the modeled Publisher-side and Subscriber-side channels.

6.4 Verification scope

The ProVerif results apply to the symbolic cryptographic model and its stated adversarial assumptions. The model treats cryptographic primitives as ideal and covers the specified secrecy and correspondence properties; it does not model implementation defects, side channels, host compromise, resource exhaustion, or ML classification behavior. Section 7 evaluates implementation performance and runtime detection experimentally.

Accordingly, the ProVerif results verify secrecy and authenticated progression across the modeled epoch transition, while the broader forward-secrecy property of the concrete protocol also depends on correct ephemeral-key generation and secure key erasure.

7 Experimental evaluation and results

This section presents the empirical evaluation of MQTT-HS. The evaluation focuses on two main dimensions: protocol-level runtime behavior and ML-IDS detection performance. The protocol-level evaluation assesses operational correctness, session reliability, cryptographic overhead, comparison with a mutual TLS baseline, runtime enforcement behavior, and host-side resource consumption. The ML-IDS evaluation assesses the three-stage hierarchical detection pipeline using stage-wise benchmarking, confusion-matrix analysis, inference latency, model size, and sensitivity analysis.

Table 5 Summary of ProVerif security verification results

Property Group	Verified Property	Query Type	Outcome
Base-secret secrecy	basePB and baseBS remain unreachable to the adversary	Secrecy query	Verified
Payload confidentiality	Protected payloads remain secret	Secrecy query	Verified
Payload non-interference	The adversary cannot distinguish protected payloads through observations	Strong secrecy / non-interference	Verified
Epoch-key secrecy	Derived keys for epoch 0 and epoch 1 remain secret	Secrecy query	Verified
Mutual authentication	Client session completion corresponds to a valid Proxy acceptance	Injective agreement	Verified
Replay protection	Replayed messages cannot produce unmatched accept events	Injective correspondence	Verified
Rekey authentication	Rekey completion corresponds to a valid endpoint rekey event	Injective agreement	Verified
Message-flow agreement	Send, receive, forward, and delivery events remain consistent across epochs	Correspondence query	Verified
Epoch-transition secrecy	Initial and refreshed epoch keys and protected payloads remain secret across the modeled key-refresh execution	Epoch-based secrecy queries	Verified

7.1 Experimental setup

MQTT-HS was evaluated in host-based and embedded environments. The Security Proxy, MQTT clients, Mosquitto Broker, cryptographic engine, and attack tools ran on a local Windows machine, while dataset preprocessing and ML-IDS training were performed in Google Colab. Endpoint performance was evaluated using an ESP32-D0WD-V3 implementation of MQTT-HS and a mutual TLS 1.2 baseline on the same device.

Unless otherwise stated, quantitative results are reported as mean ± standard deviation across three independent runs.

Both embedded configurations used the same ESP32 device, Wi-Fi network, Broker host, MQTT topic, client identifier, application-payload structure, and transmission workload. Each configuration completed three runs of 120 MQTT QoS 0 messages transmitted at 1.8-s intervals. MQTT-HS applied key refresh after messages 40 and 80, while mTLS maintained the established TLS session throughout each run.

Serial instrumentation recorded setup, cryptographic, publish-call, and rekey latencies, together with delivery status, RSSI, firmware size, and heap measurements. Subscriber logs verified unique deliveries, duplicates, missing indices, and invalid payloads. Concurrent packet capture and process-tree monitoring recorded network traffic and supporting host-resource consumption.

Table 6 summarizes the hardware, software, and workload configurations used in the evaluation.

The host baseline used MQTT v3.1.1 over mutual TLS 1.3 with the `TLS_AES_256_GCM_SHA384` cipher suite and ECDSA P-256 certificates. Each run established a full, non-resumed connection and transmitted 120 QoS 1 messages. The embedded baseline used mutual TLS 1.2 with Broker and client certificate verification. The reported mTLS connection-path measurements include TCP establishment, TLS negotiation, mutual authentication, and MQTT CONNECT/CONNACK processing.

Runtime evaluation covered benign traffic, Flood, SlowITe, and Malformed behavioral scenarios, together with protocol-level replay, untrusted-certificate, and repeated-invalid-signature scenarios.

The replay scenario retransmitted an accepted authenticated envelope ten times within an active Publisher session. The untrusted-certificate scenario submitted signed connection requests carrying self-signed certificates absent from the Proxy Trust Store. The repeated-invalid-signature scenario submitted twenty requests per run using the trusted Publisher certificate with invalid ECDSA signatures; after five verification failures, subsequent requests were rejected through temporary authentication blocking.

Each protocol-level security scenario was executed in three runs. Replay and untrusted-certificate scenarios used ten attempts per run, while the invalid-signature scenario used twenty. Measurements included rejection count and latency, session and challenge creation, ECDH/HKDF execution, downstream delivery, and legitimate recovery where applicable.

7.2 Embedded MQTT-HS evaluation on ESP32

The Publisher-side MQTT-HS implementation was deployed on an ESP32-D0WD-V3 with two CPU cores operating at 240 MHz, 4 MB of flash memory, and no external PSRAM. The implementation was compiled using ESP-IDF v5.5.5 and Arduino-ESP32 core v3.3.11 and performed certificate-based authentication, ephemeral ECDH over NIST P-256, HKDF-SHA256 key derivation, AES-256-GCM encryption, MQTT publication, and message-count-based key refresh. Three independent runs transmitted 120 messages each at a fixed interval of 1.8 s.

Across the three runs, the implementation delivered all 360 protected messages without publication failures and

Table 6 Host-based and embedded experimental configurations

Component	Configuration
Host platform	Intel Core i7-10510U, 4 cores and 8 logical processors, 16 GB RAM, Windows 11 64-bit
Runtime software	Python 3.13.1, Mosquitto 2.0.22, Paho MQTT 2.1.0, OpenSSL 3.0.15, and `cryptography` 46.0.3
ML environment	Google Colab with Ubuntu 22.04.5 LTS and Python 3.12.13
ML libraries	XGBoost 3.1.2, scikit-learn 1.7.2, and joblib 1.5.2
Host mTLS configuration	MQTT v3.1.1 over mutual TLS 1.3 using `TLS_AES_256_GCM_SHA384` and directly CA-signed ECDSA P-256 certificates
Host mTLS workload	Three runs of 120 QoS 1 messages at approximately 1.8-second intervals, with full connection establishment and no observed session reuse
Embedded platform	ESP32-D0WD-V3, dual-core CPU at 240 MHz, 4 MB flash, and no external PSRAM
Embedded software	ESP-IDF v5.5.5 with Arduino-ESP32 core v3.3.11
Embedded workload	Three runs per configuration, with 120 MQTT QoS 0 messages per run at 1.8-second intervals
Embedded MQTT-HS	ECDH over NIST P-256, HKDF-SHA256, ECDSA-SHA256 certificate authentication, AES-256-GCM payload protection, and key refresh after 40 protected messages
Embedded mTLS baseline	Mutual TLS 1.2 with Broker and client X.509 certificate verification on TCP port 8883
Serial instrumentation	Automated logging at 115200 baud for latency, delivery, heap, firmware-size, and RSSI measurements
Network capture	TShark/Wireshark PCAPNG capture on the Wi-Fi interface for frame-level and TCP-payload measurements
Host monitoring	PID-specific process-tree monitoring using `psutil` for Broker, Proxy, and Subscriber CPU and resident-memory measurements

Table 7 Embedded MQTT-HS performance on ESP32 across three independent runs

Metric	Result across three runs
Protected-message delivery	360/360
Secure setup latency	517.139 ± 2.009 ms
ECDH/HKDF key-derivation latency	151.836 ± 0.061 ms
AES-256-GCM encryption latency	0.354 ± 0.004 ms/message
MQTT publish-call latency	1.366 ± 0.002 ms/message
Rekey latency	151.968 ± 0.195 ms
Applied rekeys	2 per run
Final free heap	216, 124 ± 90 bytes
Minimum free heap	167, 491 ± 82 bytes
Largest free block	110, 580 bytes in every run
Firmware size	936, 448 bytes
Mean protected MQTT payload	611.4 bytes
RSSI	− 49.144 ± 0.518 dBm

Table 8 Host-side resource consumption during the embedded MQTT-HS runs

Component	Mean CPU (%)	Peak CPU (%)	Mean RSS (MiB)
Mosquitto Broker	0.197 ± 0.040	3.97 ± 1.67	28.613 ± 0.202
Security Proxy	1.537 ± 0.153	139.33 ± 5.54	277.443 ± 0.304
Subscriber	0.053 ± 0.035	3.00 ± 0.00	58.463 ± 0.076

applied six scheduled rekeys, with all subsequent messages delivered.

Table 7 summarizes the embedded results. ECDH/HKDF key derivation accounted for the largest measured endpoint-side cryptographic latency, while steady-state AES-256-GCM encryption remained below one millisecond per message.

Table 8 reports the supporting host-resource measurements collected during the embedded runs.

The Security Proxy used 1.537 ± 0.153% mean CPU and 277.443 ± 0.304 MiB of resident memory while loading the XGBoost hierarchy, preprocessing artifacts, session state, cryptographic material, and metadata windows. The RSS measurement covers the complete Python process and loaded runtime libraries rather than the serialized models alone. CPU peaks above 100% reflect the monitoring utility's per-logical-processor reporting convention.

7.3 Embedded comparison with mutual TLS 1.2

The embedded MQTT-HS implementation was compared with a mutual TLS 1.2 baseline on the same ESP32-D0WD-V3 platform. Both configurations used the same wireless network, Broker host, MQTT topic, application workload, client identifier, transmission interval, and number of independent runs.

Both configurations delivered all 360 application messages. MQTT-HS additionally completed six in-session rekeys with all subsequent messages delivered.

Table 9 summarizes the endpoint-side comparison.

The MQTT-HS setup path required 517.139 ± 2.009 ms, compared with 1292.036 ± 96.777 ms for the mTLS connection path. mTLS achieved lower publish-call latency and

required no additional Proxy. MQTT-HS retained 36,784 additional bytes of final free heap and 42,858 additional bytes of minimum free heap. Its firmware image was 58,208 bytes smaller, while the protocol also provided controlled rekeying, metadata-based detection, and inline enforcement.

Table 10 reports the supporting host-process resources used during the embedded runs. MQTT-HS used the Security Proxy for session management, hierarchical ML inference, metadata buffering, enforcement, decryption, and subscriber-specific re-encryption, while the mTLS configuration used the Broker and Subscriber processes. These CPU and resident-memory measurements apply to the supporting host processes rather than to the ESP32 processor. Endpoint-side evaluation quantified setup and operation latency, free heap, firmware size, delivery reliability, RSSI, and network traffic.

7.3.1 Packet-level network overhead

Packet-level network overhead was derived from six PCAPNG captures by summing captured frame lengths and TCP payload lengths. Traffic direction was identified using the ESP32 IP address. Metrics are reported across three runs per configuration, and timestamp-based attribution was used to isolate steady-state communication between the first and final application messages.

Table 11 reports the packet-level network measurements for the two embedded configurations.

MQTT-HS and mTLS generated similar packet counts, but MQTT-HS transferred more data per delivered message. Aggregate frame traffic increased from 378.70 ± 11.08 bytes/message with mTLS to 818.04 ± 43.44 bytes/message with MQTT-HS. During steady-state transmission, the corresponding values were 332.41 ± 6.14 and 765.38 bytes/message, respectively.

The additional traffic was concentrated in ESP32 transmissions and reflected the larger MQTT-HS application-layer envelope, including ciphertext, authentication data, nonces, session identifiers, and protocol-safe metadata. This overhead accompanied the Proxy-mediated design, which preserved more endpoint free heap while enabling MQTT-aware detection and enforcement.

Table 9 Embedded comparison of MQTT-HS and mutual TLS across three independent runs

Metric	MQTT-HS	mTLS
Aggregate delivery	360/360 (100%)	360/360 (100%)
Measured setup-path latency	517.139 ± 2.009 ms	1292.036 ± 96.777 ms
MQTT publish-call latency	1.366 ± 0.002 ms	1.206 ± 0.007 ms
Final free heap	216, 124 ± 90 bytes	179, 340 ± 11 bytes
Minimum free heap	167, 491 ± 82 bytes	124, 633 ± 1, 764 bytes
Largest free block	110, 580 bytes	110, 580 bytes
Firmware size	936, 448 bytes	994, 656 bytes
RSSI	− 49.144 ± 0.518 dBm	− 52.689 ± 1.921 dBm
Controlled in-session rekeys	2 per run	Not applicable

Note: MQTT-HS setup includes the authenticated request–challenge exchange and Publisher–Proxy session activation. The mTLS connection path includes TCP establishment, TLS 1.2 negotiation, mutual certificate authentication, and MQTT CONNECT/CONNACK processing

7.4 Host-based session and rekey performance

All host runs completed session establishment, Subscriber readiness, scheduled rekeying, and post-rekey delivery under the configured threshold of 40 protected messages or 300 s. The first protected delivery followed rekey completion after 0.881 ± 0.161 ms on average.

7.5 Host-based protocol performance evaluation

This subsection reports implementation microbenchmarks collected from the Python MQTT-HS prototype on the local Intel Core i7 platform. Figure 7 presents the setup-related and per-message cryptographic timings.

The host-based Publisher handshake required 7.395 ± 0.910 ms, while Publisher-side and Proxy-side key derivation required 0.256 ± 0.037 ms and 0.239 ± 0.055 ms, respectively. Per-message costs were 0.088 ± 0.002 ms for Publisher encryption, 0.081 ± 0.012 ms for Proxy decryption, 0.039 ± 0.004 ms for Proxy re-encryption, and 0.056 ± 0.010 ms for Subscriber decryption. The corresponding ESP32 measurements are reported in Section 7.2.

Table 10 Supporting host resources during the embedded MQTT-HS and mTLS runs

Component	Mean CPU (%)		Mean RSS (MiB)	
	MQTT-HS	mTLS	MQTT-HS	mTLS
Broker	0.197 ± 0.040	0.153 ± 0.029	28.613 ± 0.202	31.497 ± 0.055
Security Proxy	1.537 ± 0.153	N/A	277.443 ± 0.304	N/A
Subscriber	0.053 ± 0.035	0.053 ± 0.025	58.463 ± 0.076	53.400 ± 0.095

Table 11 Packet-level network overhead of embedded MQTT-HS and mTLS

Metric	MQTT-HS	mTLS
Packet count per run	282.3 ± 17.9	278.0 ± 6.1
Total frame bytes per run	98, 164.7 ± 5, 212.9	45, 443.7 ± 1, 329.2
ESP32 TX frame bytes per run	86, 681.3 ± 2, 998.8	35, 727.7 ± 530.0
ESP32 RX frame bytes per run	11, 483.3 ± 2, 214.1	9, 716.0 ± 831.6
Aggregate frame bytes/message	818.04 ± 43.44	378.70 ± 11.08
Steady-state frame bytes/message	765.38 in every run	332.41 ± 6.14
Steady-state TCP payload bytes/message	652.43 in every run	219.91 ± 4.23
Average frame bit rate	3, 575.98 ± 64.41 bit/s	1, 637.81 ± 44.95 bit/s

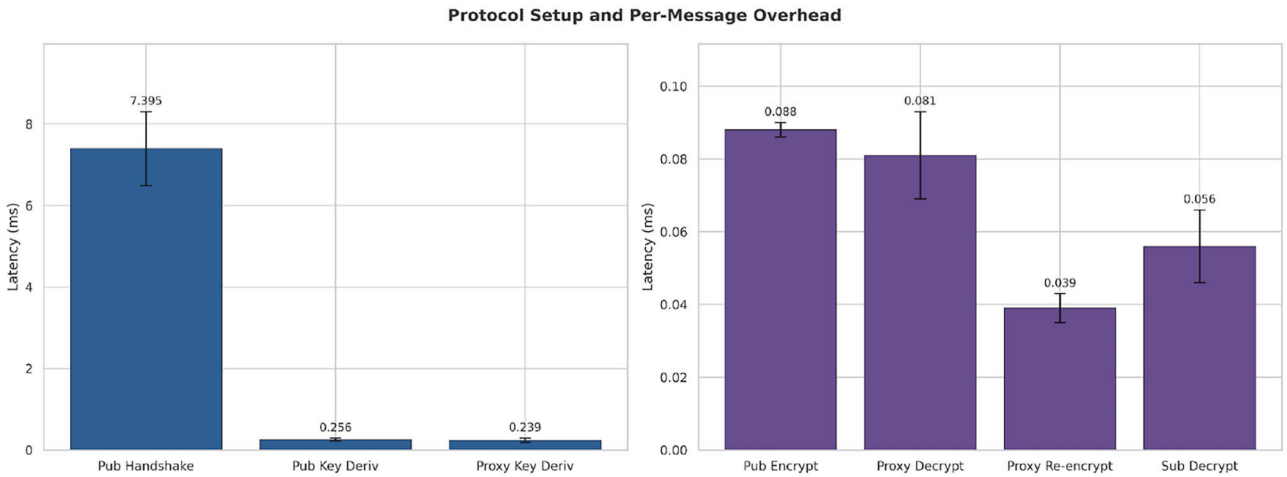


Fig. 7 Host-based setup and per-message cryptographic latencies of MQTT-HS

7.6 Host-based comparison with mutual TLS 1.3

The host-based baseline used the mutual TLS 1.3 configuration described in Section 7.1. Three independent runs transmitted 120 QoS 1 messages each without observed TLS session reuse. The mTLS connection-path measurement included TCP establishment, TLS negotiation, mutual certificate authentication, and MQTT CONNECT/CONNACK processing.

The MQTT-HS Publisher reached authenticated session readiness in 7.395 ± 0.910 ms, whereas the evaluated mTLS connection path required 12.932 ± 10.146 ms. The individual mTLS measurements were 6.868, 7.283, and 24.646 ms, indicating greater between-run variation in the mTLS measurements.

The measured mTLS end-to-end delivery latency was 1.988 ± 0.017 ms. MQTT-HS performs additional Proxy-side decryption, metadata processing, ML inference, enforcement, and Subscriber-specific re-encryption; its internal-stage timings are therefore reported separately.

Figure 8 compares the host-based cryptographic and rekey-operation latencies of MQTT-HS.

Rekey application produced the highest mean internal operation latency, reaching 0.959 ± 0.209 ms on the Publisher and 0.609 ± 0.589 ms on the Subscriber. The remaining measured cryptographic operations required less than 0.3 ms on average.

The host-based mTLS configuration completed all three connections and delivered all 360 transmitted messages. During these runs, the Mosquitto Broker used 0.104 ± 0.015% mean CPU and 13.531 ± 0.346 MiB of resident memory. These resource measurements cover the Broker process only.

7.7 Runtime enforcement under benign and attack traffic

In the runtime scenarios, the benign trace allowed all evaluated messages, while the Flood, SlowITe, and Malformed traces blocked all evaluated attack messages.

Figure 9 compares runtime decision latency across the four traffic scenarios.

Mean decision latency ranged from 6.195 ± 0.380 ms for benign traffic to 11.764 ± 2.157 ms for SlowITe. Flood and Malformed traffic required 10.290 ± 0.665 ms and 6.884 ± 0.810 ms, respectively.

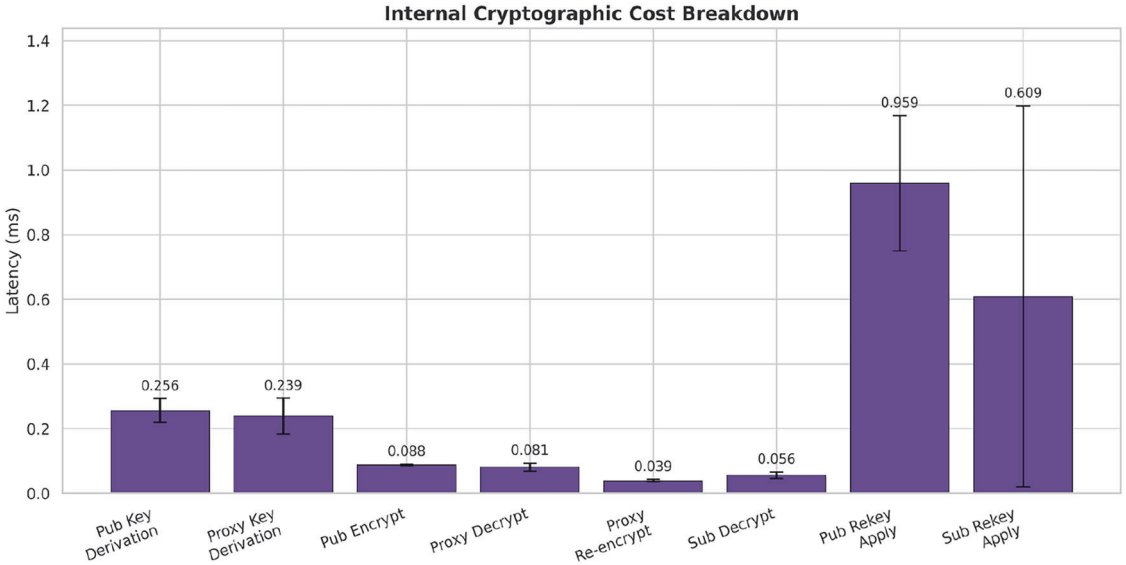


Fig. 8 Host-based cryptographic and rekey-operation latencies of MQTT-HS

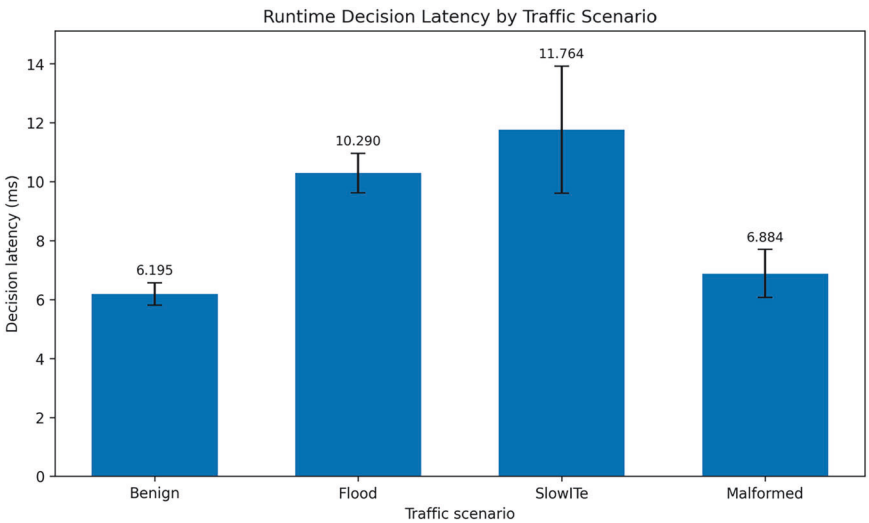


Fig. 9 Runtime decision latency under benign and attack traffic

7.8 Protocol-level enforcement against replay and authentication abuse

Protocol-level enforcement was evaluated against ciphertext replay, untrusted certificates, and repeated invalid-signature attempts. These scenarios were processed before or independently of hierarchical ML-IDS inference and were executed in three runs each.

Replay attempts were rejected without entering the ML window, incrementing the message counter, triggering rekey evaluation, or reaching the Subscriber. Untrusted certificates were rejected before challenge issuance and session-key derivation. In the repeated-invalid-signature scenario, the first five attempts per run underwent signature verification, after which temporary blocking rejected the remaining attempts. Legitimate authentication resumed after block expiry in all three runs.

Table 12 summarizes the rejection counts, enforcement paths, and measured rejection latencies for the three protocol-level scenarios.

7.9 Host-side runtime resource consumption

Across the benign and attack scenarios, mean CPU utilization ranged from 4.7% to 5.5%, while mean resident memory remained between 248 and 249 MiB. Peak

Table 12 Protocol-level enforcement against replay and authentication abuse

Scenario	Attempts	Rejected	Enforcement result
Ciphertext replay	30	30	No replay delivery; rejection latency of 0.0047 ± 0.0027 ms
Untrusted certificate	30	30	Rejected before challenge issuance, session creation, and ECDH/HKDF; 0.1381 ± 0.0846 ms
Invalid signatures	60	60	15 signature-verification rejections at 1.1731 ± 1.6606 ms and 45 temporary-block rejections at 0.1700 ± 0.1093 ms; legitimate recovery in 3/3 runs

Table 13 Stage 0 benchmark results

Model	Acc	B-Acc	m-F1	w-F1	m-Prec	m-Rec	Train	Inf.	T-put	Size
XGBoost	0.973	0.808	0.839	0.972	0.878	0.808	7.42	0.017	5.94×10^4	0.72
Random Forest	0.974	0.754	0.819	0.970	0.938	0.754	9.15	0.085	1.17×10^4	15.21
Decision Tree	0.940	0.710	0.696	0.942	0.684	0.710	0.98	0.001	1.14×10^6	0.06
LSTM	0.825	0.773	0.592	0.871	0.580	0.773	17.59	0.400	2.50×10^3	0.30
GRU	0.822	0.766	0.588	0.868	0.577	0.766	27.08	0.417	2.40×10^3	0.25
BiLSTM	0.776	0.769	0.558	0.838	0.566	0.769	28.38	0.664	1.50×10^3	0.58
MLPFlat	0.755	0.774	0.546	0.824	0.564	0.774	6.92	0.143	7.01×10^3	1.70
CNN1D-GAP	0.617	0.777	0.472	0.722	0.552	0.777	14.97	0.201	4.98×10^3	0.25

Acc accuracy, *B-Acc* balanced accuracy, *m-F1* macro F1-score, *w-F1* weighted F1-score, *m-Prec* macro precision, *m-Rec* macro recall, *Train* training time in seconds, *Inf.* inference time per sample in milliseconds, *T-put* throughput in samples per second, *Size* serialized model size in MB

memory remained close to the corresponding scenario means, with no scenario-specific increase during the measured runs. CPU peaks reflected transient execution across multiple logical processors.

7.10 Machine learning-based IDS evaluation

This subsection evaluates the three-stage hierarchical ML-IDS. Stage 0 distinguishes Legitimate from Attack traffic, Stage 1 assigns detected attacks to their corresponding families, and Stage 2A distinguishes DoS from SlowITe within the Denial-Family. The evaluation covers predictive performance, training time, inference latency, throughput, and serialized model size.

7.10.1 Stage 0 benchmarking and model selection

Stage 0 performs the initial Legitimate-versus-Attack classification. Model selection considered balanced accuracy, macro F1-score, class-wise performance, inference latency, throughput, and serialized model size. Table 13 reports the complete Stage 0 benchmark results.

XGBoost achieved the highest balanced accuracy and macro F1-score among the evaluated Stage 0 models, with 0.017 ms inference latency and a serialized size of 0.72 MB. Random Forest achieved similar overall accuracy but lower balanced accuracy and a substantially larger model size of 15.21 MB.

Figure 10 presents the confusion matrix of the selected Stage 0 XGBoost model.

Stage 0 correctly identified 1714 of 1730 attack windows, yielding an attack recall of 99.075%. It classified 33 of 88 legitimate windows as attacks, producing a false-positive rate of 37.500%. The Wilson 95% confidence interval was 28.108%–47.938% for the false-positive rate and 98.503%–99.430% for attack recall. The wider false-positive interval reflects the smaller number of legitimate holdout windows. Because the windows overlap at a stride of 20 events, adjacent observations are not fully independent. The reported Wilson intervals should therefore be interpreted as descriptive uncertainty bounds for this holdout configuration rather than as complete estimates of deployment-level false-alarm uncertainty. A Stage 0 false-positive classification routes a legitimate window into the attack-classification and enforcement path.

This operational effect is considered further in the discussion of deployment implications.

7.10.2 Stage 1 benchmarking and attack-family attribution

Stage 1 assigns windows classified as attacks to Bruteforce, Flood, Malformed, or Denial-Family. Table 14 reports the Stage 1 benchmark results.

XGBoost was selected for Stage 1 because it maintained balanced accuracy comparable to Random Forest (0.960 versus 0.963), while reducing inference latency from 0.075 to 0.023 ms and model size from 19.95 to 2.32 MB.

Figure 11 presents the confusion matrix of the selected Stage 1 model. Bruteforce and Flood windows were

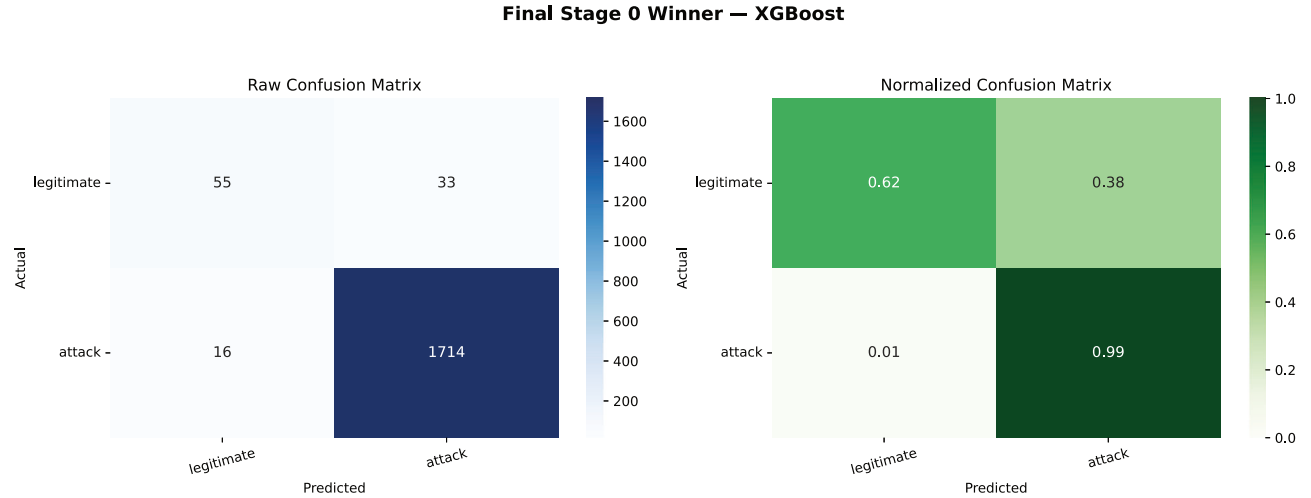


Fig. 10 Confusion matrix of the selected Stage 0 XGBoost model

Table 14 Stage 1 benchmark results

Model	Acc	B-Acc	m-F1	w-F1	m-Prec	m-Rec	Train	Inf.	T-put	Size
XGBoost	0.927	0.960	0.952	0.928	0.947	0.960	23.50	0.023	4.32×10^4	2.32
Random Forest	0.923	0.963	0.950	0.925	0.945	0.963	10.56	0.075	1.33×10^4	19.95
LSTM	0.906	0.958	0.940	0.908	0.936	0.958	48.92	0.417	2.40×10^3	0.31
GRU	0.906	0.958	0.940	0.908	0.936	0.958	86.20	0.436	2.29×10^3	0.25
BiLSTM	0.905	0.958	0.940	0.908	0.936	0.958	87.03	0.766	1.31×10^3	0.58
CNN1D-GAP	0.903	0.954	0.938	0.906	0.934	0.954	17.09	0.187	5.36×10^3	0.25
MLPFlat	0.897	0.953	0.929	0.900	0.921	0.953	5.34	0.143	6.97×10^3	1.70
Decision Tree	0.898	0.919	0.921	0.897	0.924	0.919	1.35	0.001	1.13×10^6	0.07

Acc accuracy, *B-Acc* balanced accuracy, *m-F1* macro F1-score, *w-F1* weighted F1-score, *m-Prec* macro precision, *m-Rec* macro recall, *Train* training time in seconds, *Inf.* inference time per sample in milliseconds, *T-put* throughput in samples per second, *Size* serialized model size in MB

classified correctly, while the remaining errors were concentrated between Malformed and Denial-Family traffic.

7.10.3 Stage 2A representation ablation and DoS/SlowITe discrimination

Stage 2A distinguishes DoS from SlowITe within the Denial-Family. Table 15 compares the Direct-11, Augmented-14, and Pruned-10 feature representations across XGBoost and CNN1D-GAP models.

Pruned-10 XGBoost achieved the highest Stage 2A balanced accuracy and macro F1-score, with 0.010 ms inference latency and a 0.43 MB model size.

Figure 12 shows that the selected model correctly classified 323 of 339 DoS windows and 105 of 136 SlowITe windows.

7.10.4 Pipeline sensitivity and mixed-window behavior

Post-hoc analyses examined window size, purity threshold, feature groups, mixed-window behavior, warm-up duration, and hierarchical inference cost. Each Stage 0 ablation reconstructed the windows, fitted preprocessing on the training loops, retrained XGBoost using the same hyperparameters, and evaluated the resulting model on Loop 2. These analyses were conducted separately from the original three-stage model benchmarks.

The window-size ablation evaluated $W \in \{20, 40, 60\}$ to examine the effect of shorter and longer behavioral contexts around the implemented setting of $W = 40$. The stride was set to half the window size, while the feature representation, XGBoost hyperparameters, loop-level split, and purity threshold remained unchanged.

Table 16 reports the Stage 0 results obtained after reconstructing and retraining each window-size configuration.

Larger windows increased attack recall and macro F1-score by capturing a broader behavioral context, but also required more events before each decision. Accordingly, $W = 40$ and $S = 20$ were used to balance behavioral context, decision frequency, and warm-up requirements in the runtime pipeline.

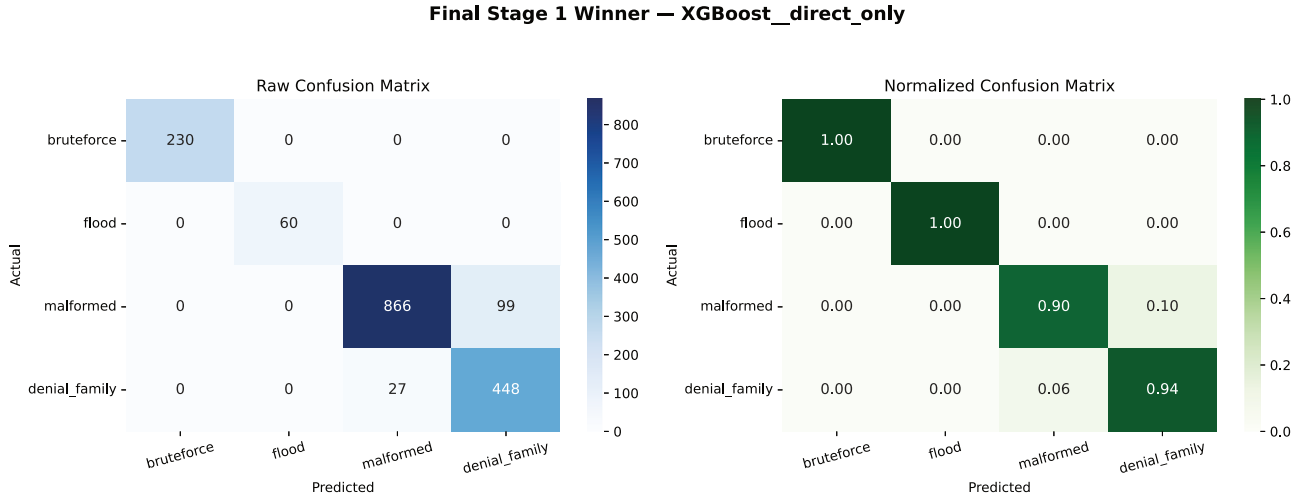


Fig. 11 Confusion matrix of the selected Stage 1 XGBoost model

Table 15 Stage 2A representation and model ablation results

Representation	Model	Acc	B-Acc	m-F1	w-F1	Train	Inf.	Size
Pruned-10	XGBoost	0.901	0.862	0.875	0.899	3.13	0.010	0.43
Direct-11	XGBoost	0.897	0.853	0.868	0.894	1.90	0.010	0.43
Direct-11	CNN1D-GAP	0.901	0.829	0.864	0.894	10.42	0.492	0.25
Pruned-10	CNN1D-GAP	0.899	0.830	0.862	0.893	8.64	0.706	0.25
Augmented-14	XGBoost	0.888	0.847	0.859	0.886	3.00	0.010	0.43
Augmented-14	CNN1D-GAP	0.886	0.815	0.845	0.879	7.79	0.782	0.26

Acc accuracy, *B-Acc* balanced accuracy, *m-F1* macro F1-score, *w-F1* weighted F1-score, *Train* training time in seconds, *Inf.* inference time per sample in milliseconds, *Size* serialized model size in MB

Mixed windows in the purity analysis were assigned their majority label. The resulting labels were used to examine sensitivity to the purity threshold rather than as unambiguous transition ground truth.

Table 17 reports the sensitivity of Stage 0 to the window-purity threshold at $W = 40$.

A purity threshold of 0.90 produced a modest improvement over 1.0, whereas reducing it to 0.75 increased the false-positive rate and reduced macro F1-score. The primary pipeline used a threshold of 1.0 to preserve unambiguous labels, while mixed windows were analyzed separately as transition cases.

Among 301 mixed Loop 2 windows, 91.36% were classified as attacks. Detection increased from approximately 60.7% for windows with less than 10% attack occupancy to 98.8% for windows with 90%–100% attack occupancy. These values describe transition sensitivity under different levels of attack occupancy rather than single-class classification accuracy.

Feature-group ablation showed the largest degradation after removing message-length features, reducing balanced accuracy by 22.48 percentage points and specificity by 44.31 percentage points. Removing the timing, MQTT-control, session, or transport feature groups produced smaller changes.

Warm-up duration varied with traffic rate. Legitimate Loop 2 windows required a median of 0.29 s and a 95th percentile of 1.28 s to accumulate 40 events, compared with 2.16 s and 18.63 s for SlowITe. After warm-up, the stride of $S = 20$ produces a decision every 20 additional accepted events.

The original selected models required 0.017 ms for Stage 0, 0.040 ms through Stage 1, and 0.050 ms for the complete three-stage model-inference path. These measurements exclude feature construction, cryptographic processing, logging, and enforcement.

7.10.5 Final hierarchical model summary

XGBoost was selected for all three stages. Stage 0 and Stage 1 use the Direct-11 representation, while Stage 2A uses Pruned-10. Table 18 summarizes the selected models.

The three selected models occupied 3.47 MB in total and required 0.050 ms for the complete three-stage model-inference path.

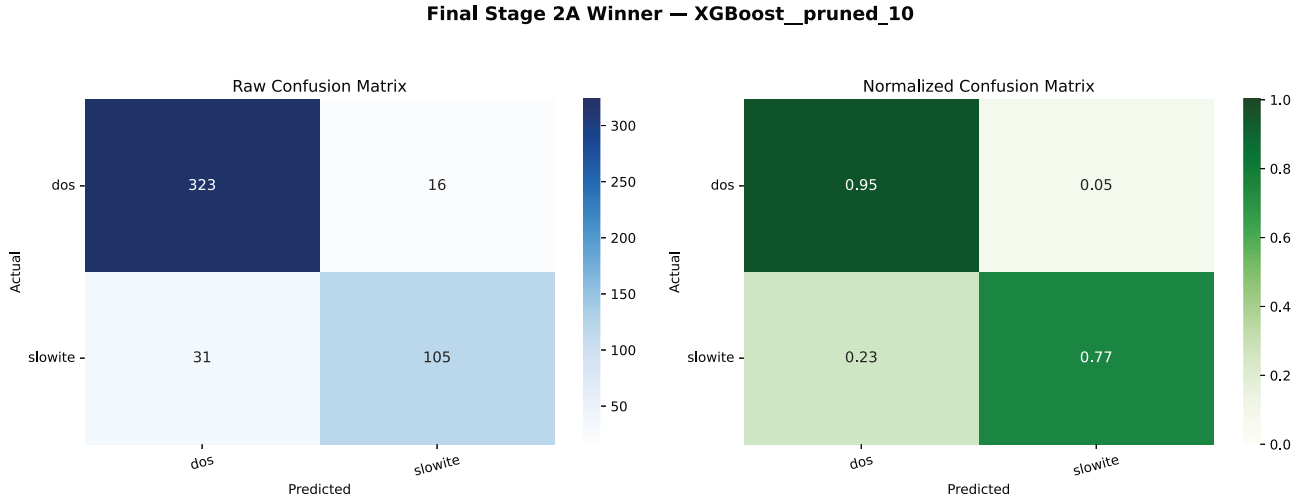


Fig. 12 Confusion matrix of the selected Stage 2A XGBoost model

Table 16 Stage 0 sensitivity to the sliding-window size

Win.	Stride	Legit.	Attack	FPR	Spec.	Recall	B-Acc	M-F1
20	10	226	3696	32.301	67.699	97.998	82.848	82.780
40	20	88	1730	37.500	62.500	99.017	80.759	83.656
60	30	49	1073	36.735	63.265	99.441	81.353	85.490

FPR false-positive rate, *Spec.* specificity, *Recall* attack recall, *B-Acc* balanced accuracy, *M-F1* macro F1-score. All performance values are percentages. Every configuration used the Direct-11 representation, identical XGBoost hyperparameters, the same loop-level split, a purity threshold of 1.0, and a stride equal to half the window size. The $W = 40$ row was produced by the consistent ablation retraining pipeline and differs slightly from the separately saved final Stage 0 model

Table 17 Stage 0 sensitivity to the window-purity threshold at $W = 40$

Purity	Retained	Mixed	FPR	Recall	B-Acc	M-F1
1.00	9,574	0	37.500	99.017	80.759	83.656
0.90	10,283	709	**36.458**	**99.196**	**81.369**	**84.798**
0.75	10,794	1,220	42.609	98.713	78.052	81.092

FPR false-positive rate, *Recall* attack recall, *B-Acc* balanced accuracy, *M-F1* macro F1-score. Performance values are percentages. Mixed windows were assigned their majority label for this sensitivity analysis. Bold values indicate the best observed performance for each metric among the purity thresholds compared in the table: the lowest false-positive rate (FPR) and the highest attack recall, balanced accuracy (B-Acc), and macro F1-score (M-F1)

8 Discussion and limitations

8.1 Discussion of main findings

The evaluation provides prototype-level evidence that MQTT-HS can combine authenticated payload protection, controlled key refresh, metadata-based detection, and inline enforcement. On the ESP32, steady-state AES-256-GCM encryption remained sub-millisecond, while ECDH/HKDF setup and rekey operations required approximately 152 ms and represented the dominant endpoint-side cryptographic costs.

The mTLS comparison revealed different cost distributions rather than uniform superiority. MQTT-HS retained more ESP32 free heap, produced a smaller evaluated firmware image, and added MQTT-aware detection and enforcement. mTLS achieved lower publish-call latency, substantially lower network overhead, and required no additional Security Proxy. MQTT-HS therefore trades additional network and Proxy-side cost for broader application-aware security functionality.

Replay, untrusted-certificate, and repeated-invalid-signature attempts were handled through protocol-level controls rather than ML classification. Flood, SlowITe, and Malformed traffic were evaluated through metadata-based behavioral detection. This separation reduces dependence on a single enforcement mechanism but does not extend the controlled runtime results to untested attacks or deployment conditions.

Stage 0 achieved 99.075% attack recall, alongside a 37.5% false-positive rate estimated from 88 legitimate holdout windows. The observed false-positive behavior indicates that persistent blocking may require confidence-aware or consecutive-window confirmation policies, which

Table 18 Selected models for the hierarchical ML-IDS

Stage	Model	Representation	Acc	B-Acc	m-F1	Inf.	Size
Stage 0	XGBoost	Direct-11	0.973	0.808	0.839	0.017	0.72
Stage 1	XGBoost	Direct-11	0.927	0.960	0.952	0.023	2.32
Stage 2A	XGBoost	Pruned-10	0.901	0.862	0.875	0.010	0.43

Acc accuracy, *B-Acc* balanced accuracy, *m-F1* macro F1-score, *Inf.* inference time per sample in milliseconds, *Size* serialized model size in MB

Table 19 Literature-based architectural comparison of representative MQTT security frameworks

Study	Crypto. Prot.	Formal Verif.	Integrated ML-IDS	Act. Inline Enforce.	Forward Secrecy
Alharbi et al. (2025)	Yes	No	No	Partial	No
Gao et al. (2025)	Yes	Yes	No	No	Yes
Karimunda et al. (2025)	Yes	No	Yes	No	No
Alhossainy & Malekzadeh (2026)	No	No	Yes	No	No
Qasem et al. (2026)	Yes	No	Yes	No	No
MQTT-HS (Proposed)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

Note: Entries describe capabilities reported in the cited publications. "Yes", "No", and "Partial" indicate architectural coverage rather than equivalent implementation depth or performance

were not evaluated in the present prototype. Warm-up duration also depends on traffic rate, and mixed low-occupancy transitions remain less certain than attack-dominant windows. These findings support prototype feasibility while motivating broader benign and cross-environment validation.

8.2 Comparative evaluation against the state of the art

Prior MQTT security studies address different portions of the problem, including cryptographic protection, formal verification, intrusion detection, and inline enforcement. Table 19 compares the architectural capabilities reported by representative related studies.

The comparison is literature-based rather than experimental. The referenced systems were not reimplemented under a common hardware, dataset, traffic, or measurement configuration; therefore, the table characterizes reported functional coverage rather than quantitative performance.

MQTT-HS combines all five listed functions within the evaluated prototype. This broader functional coverage does not establish quantitative superiority over the referenced systems. The controlled quantitative comparison in this study is limited to the implemented mutual TLS baselines described in Section 7.1.

8.3 Deployment implications

The MQTT-HS architecture has the following deployment implications:

- **Workload placement:** ML inference, policy enforcement, and subscriber-specific re-encryption are shifted to the Security Proxy, reducing endpoint responsibilities while introducing Proxy-side resource cost.
- **MQTT compatibility:** Publishers remain independent of Subscriber identities, and the Broker retains its standard routing role.
- **Metadata-based classification:** The ML-IDS does not use plaintext payload content, although the trusted Proxy processes plaintext transiently for subscriber-specific re-encryption.
- **Gateway placement:** The architecture is intended for an edge gateway or IoT concentrator before the Broker. Broader suitability requires multi-client, redundancy, and cross-environment evaluation.

The centralized Proxy is both an enforcement boundary and a single point of dependency. Under a fail-closed policy, Proxy unavailability interrupts new handshakes, protected forwarding, ML decisions, re-encryption, and key refresh. A fail-open policy would preserve availability by bypassing the security boundary and was not adopted by the prototype.

Proxy compromise may expose current-epoch keys, transient plaintext, authorization mappings, and enforcement state. Practical deployments should therefore apply process isolation, least privilege, protected key storage, restricted memory dumps, and secure key erasure.

Redundancy could be provided through active-passive or active-active Proxy instances, but this requires authenticated health checking, consistent authorization data, secure

session-state replication, and coordinated key epochs. Such mechanisms were not implemented in the evaluated prototype.

8.4 Scalability considerations

The Proxy-mediated design separates Publisher cost from Subscriber fan-out. A Publisher encrypts and transmits each application message once, regardless of the number of authorized Subscribers. However, the Security Proxy performs subscriber-specific encryption and forwarding for every matching Subscriber. If N authorized Subscribers receive a topic, the approximate per-message Proxy cost can be expressed as:

$$T_{\text{Proxy}}(N) \approx T_{\text{metadata}} + T_{\text{IDS}} + T_{\text{decrypt}} + N\left(T_{\text{reencrypt}} + T_{\text{forward}}\right). \tag{5}$$

As expressed in Equation (5), the Publisher-side cost is approximately independent of N, whereas Proxy-side encryption, output bandwidth, and delivery-state processing grow approximately linearly with Subscriber fan-out. Proxy memory consumption also increases with the number of active Publisher and Subscriber sessions, session keys, replay-protection states, topic mappings, sliding-window buffers, and pending output messages.

The present evaluation used one Publisher, one Subscriber, one Broker, and one Security Proxy. It therefore does not establish large-scale fan-out capacity, concurrent-session throughput, or overload tolerance. Under higher load, Subscriber-specific encryption and forwarding queues may increase latency, memory consumption, and back-pressure. Dedicated multi-client stress testing is required to quantify these effects.

8.5 Limitations

The findings of this study should be interpreted within the following limitations:

- **Trusted Proxy dependency:** MQTT-HS relies on the Security Proxy for session management, behavioral enforcement, and Subscriber-specific re-encryption. The evaluated prototype did not implement Proxy redundancy or failover.
- **ML evaluation scope:** The ML-IDS was developed and evaluated using MQTTEEB-D with a loop-level split. The Stage 0 holdout contained 88 pure legitimate windows and 1730 attack windows, limiting the precision of the legitimate-class false-positive estimate. Cross-dataset and cross-environment generalization were not evaluated.
- **Window-based detection:** Behavioral inference requires 40 authenticated events before the first complete-window decision. During this interval, protocol-level validation remains active, but behavioral decisions remain pending. Detection timing therefore depends on traffic rate, and mixed transition windows do not provide unambiguous single-class ground truth.
- **Scalability scope:** The evaluated topology used one Publisher, one Subscriber, one Broker, and one Security Proxy. Large Subscriber fan-out, concurrent multi-client workloads, and Proxy overload behavior were not experimentally evaluated.
- **Attack and fault coverage:** Runtime experiments covered the reported behavioral and protocol-level scenarios. Coordinated distributed attacks, compromised endpoints or Brokers, and packet-loss injection during individual rekey control exchanges were not evaluated.

8.6 Future work

Future work will focus on the following directions:

- distributed Proxy operation with secure state replication, coordinated key epochs, and failover evaluation;
- multi-client and multi-topic scalability testing under large Subscriber fan-out, bounded queues, and Proxy overload;
- broader validation across heterogeneous constrained devices, network conditions, Broker configurations, and representative benign workloads;
- confidence-aware and consecutive-window enforcement policies for reducing false-positive disruption;
- transition-aware modeling for mixed and low-occupancy attack windows.

9 Conclusion

This paper presented MQTT-HS, a proxy-mediated MQTT security protocol that combines authenticated session establishment, AES-256-GCM payload protection, controlled key refresh, metadata-based hierarchical intrusion detection, and inline enforcement. ProVerif verified the targeted secrecy, authentication, replay-resistance, message-flow, and epoch-key secrecy properties of the modeled cryptographic core, while the forward-secrecy argument additionally relies on fresh ephemeral material and secure key erasure.

The ESP32 prototype delivered all 360 protected messages, with sub-millisecond steady-state AES-256-GCM encryption and higher bounded costs for ECDH/HKDF setup and rekeying. Controlled runtime experiments rejected all evaluated replay, untrusted-certificate, and

repeated-invalid-signature attempts. The hierarchical XGBoost pipeline provided low model-inference latency, but Stage 0 exhibited a substantial false-positive burden on a small legitimate holdout subset.

The comparison with mTLS demonstrated a security-functionality trade-off. MQTT-HS preserved more endpoint free heap and added MQTT-aware detection, controlled rekeying, and inline enforcement, while mTLS achieved lower publish-call latency, lower network overhead, and required no additional Proxy. The results therefore support prototype feasibility under the evaluated configuration rather than confirmed production readiness. Broader benign validation, distributed Proxy operation, and multi-client scalability testing remain necessary.

Acknowledgements The authors would like to acknowledge the Deanship of Graduate Studies and Scientific Research, Taif University for funding this work.

Author contributions M.M.A. conceptualized the protocol framework, engineered the feature schemas, performed the machine learning training, conducted the formal ProVerif verification, and drafted the manuscript. M.R.A. supervised the research, refined the security architecture, reviewed the formal symbolic analysis, and thoroughly edited the final manuscript.

Funding This research was funded by the Deanship of Graduate Studies and Scientific Research, Taif University, Saudi Arabia.

Data Availability The dataset used in this study is publicly available (MQTTEEB-D dataset).

Code availability The implementation code for the MQTT-HS Security Proxy and the trained machine learning models are available from the corresponding author upon reasonable request.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Alashjaee AM, Alqahtani F (2025) Enhanced intrusion detection system IoT network security model by feed forward neural network and machine learning. Sci Rep. 15(1) https://doi.org/10.1038/s41598-025-20047-0

Alharbi S, Awad W, Bell D (2025) Hecs4mqtt: A multi-layer security framework for lightweight and robust encryption in healthcare IoT communications. Future Internet 17(7):298. https://doi.org/10.3390/fi17070298

Alhossainy AK, Malekzadeh M (2026) AI-powered threat detection framework for security enhancement of MQTT-based IoT networks. Int Trans Electr Eng Comput Sci 5(1):1–16. https://doi.org/10.62760/iteecs.5.1.2026.171

Alkhafajee AR, Al-Muqarm AMA, Alwan AH, Mohammed ZR (2021) Security and performance analysis of MQTT protocol with TLS in IoT networks. In: 2021 4th International Iraqi Conference on Engineering Technology and Their Applications (IICETA), pp 206–211. https://doi.org/10.1109/iiceta51758.2021.9717495

Al Hanif A, Ilyas M (2024) Effective feature engineering framework for securing MQTT protocol in IoT environments. Sensors 24(6):1782. https://doi.org/10.3390/s24061782

Aloufi KS, Alhazmi OH (2020) A hybrid IoT security model of MQTT and UMA. Commun Netw 12(04):155–173. https://doi.org/10.4236/cn.2020.124008

Aqachtoul A, Karam K, Elamrani A, Najib M, Rafalia N, Bakhouya M (2025) MQTTEEB-D: A real-world IoT cybersecurity dataset for AI-powered threat detection in MQTT networks. Data Brief 62:111897. https://doi.org/10.1016/j.dib.2025.111897

Bankó MB, Dyszewski S, Králová M, Limpek MB, Papaioannou M, Choudhary G, Dragoni N (2025) Advancements in machine learning-based intrusion detection in IoT: Research trends and challenges. Algorithms 18(4):209. https://doi.org/10.3390/a18040209

Choudhary A (2024) Internet of things: a comprehensive overview, architectures, applications, simulation tools, challenges and future directions. Discov Internet Things 4(1) https://doi.org/10.1007/s43926-024-00084-3

Chui KT, Gupta BB, Liu J, Arya V, Nedjah N, Almomani A, Chaurasia P (2023) A survey of internet of things and cyber-physical systems: Standards, algorithms, applications, security, challenges, and future directions. Information 14(7):388. https://doi.org/10.3390/info14070388

Dinculeană D, Cheng X (2019) Vulnerabilities and limitations of MQTT protocol used between IoT devices. Appl Sci 9(5):848. https://doi.org/10.3390/app9050848

Gao C, Xi X, Zhang C (2025) Security model design and formal verification of MQTT protocol. Discov Appl Sci 7(11) https://doi.org/10.1007/s42452-025-07749-w

Gavriilidis NO, Halkidis ST, Petridou S (2025) Empirical evaluation of TLS-enhanced MQTT on IoT devices for V2X use cases. Appl Sci 15(15):8398. https://doi.org/10.3390/app15158398

Guşiţă B, Anton AA, Stângaciu CS, Stănescu D, Găină LI, Micea MV (2025) Securing IoT edge: a survey on lightweight cryptography, anonymous routing and communication protocol enhancements. Int J Inf Secur. 24(3) https://doi.org/10.1007/s10207-025-01071-7

Husnain M, Hayat K, Cambiaso E, Fayyaz UU, Mongelli M, Akram H, Ghazanfar Abbas S, Shah GA (2022) Preventing MQTT vulnerabilities using IoT-enabled intrusion detection system. Sensors 22(2):567. https://doi.org/10.3390/s22020567

Imran, Zuhairi MF, Ali SM, Shahid Z, Alam MM, Su'ud MM (2024) Realtime feature engineering for anomaly detection in IoT-based MQTT networks. IEEE Access 12:25700–25718. https://doi.org/10.1109/access.2024.3363889

Im Y, Lim M (2023) E-mqtt: End-to-end synchronous and asynchronous communication mechanisms in mqtt protocol. Appl Sci 13(22):12419. https://doi.org/10.3390/app132212419

Kant D, Johannsen A, Creutzburg R (2021) Analysis of IoT security risks based on the exposure of the MQTT protocol. Electron

Imaging 33(3):96–1968. https://doi.org/10.2352/issn.2470-1173.2021.3.mobmu-096

Karimunda K, Ufitikirezi JDDM, Bumbálek R, Zoubek T, Bartoš P, Kuneš R, Umurungi SN, Chukwunyere A, Norbelt M, Bo G (2025) Machine learning-assisted cryptographic security: A novel ecc-ann framework for mqtt-based iot device communication. Computation 13(10):227. https://doi.org/10.3390/computation13100227

Khan MA, Khan MA, Jan SU, Ahmad J, Jamal SS, Shah AA, Pitropakis N, Buchanan WJ (2021) A deep learning-based intrusion detection system for MQTT-enabled IoT. Sensors 21(21):7016. https://doi.org/10.3390/s21217016

Kikissagbe BR, Adda M (2024) Machine learning-based intrusion detection methods in IoT systems: A comprehensive review. Electronics 13(18):3601. https://doi.org/10.3390/electronics13183601

Laghari SUA, Li W, Manickam S, Nanda P, Al-Ani AK, Karuppayah S (2024) Securing MQTT ecosystem: Exploring vulnerabilities, mitigations, and future trajectories. IEEE Access 12:139273–139289. https://doi.org/10.1109/access.2024.3412030

Mishra B, Kertesz A (2020) The use of MQTT in M2M and IoT systems: A survey. IEEE Access 8:201071–201086. https://doi.org/10.1109/access.2020.3035849

OASIS Open MQTT Version 5.0 | OASIS Standard — docs.oasis-open.org. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html. [Accessed 15-05-2026] (2019)

OASIS Open MQTT Version 3.1.1 Errata 01 — docs.oasis-open.org. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/errata01/os/mqtt-v3.1.1-errata01-os.html. [Accessed 15-05-2026] (2015)

Prantl T, Iffländer L, Herrnleben S, Engel S, Kounev S, Krupitzer C (2021) Performance impact analysis of securing MQTT using TLS. In: Proceedings of the ACM/SPEC International Conference on Performance Engineering. ICPE '21, pp 241–248. https://doi.org/10.1145/3427921.3450253

Qasem MA, Motiram BM, Thorat S, Al-Hejri AM, Alshamrani SS, Alshmrany KM (2026) Enhancement of cryptography algorithms for security of cloud-based IoT with machine learning models. Sci Rep. 16(1) https://doi.org/10.1038/s41598-026-45938-8

Reis MJCS (2025) AI-driven anomaly detection for securing IoT devices in 5G-enabled smart cities. Electronics 14(12):2492. https://doi.org/10.3390/electronics14122492

Sahota J, Vlajic N (2021) Mozi IoT malware and its botnets: From theory to real-world observations. In: 2021 International Conference on Computational Science and Computational Intelligence (CSCI), pp 698–703. https://doi.org/10.1109/csci54926.2021.00181

Salih S, Abdelmaboud A, Husain O, Motwakel A, Elshafie H, Sharif M, Hamdan M (2025) IoT in urban development: insight into smart city applications, case studies, challenges, and future prospects. PeerJ Comput Sci 11:2816. https://doi.org/10.7717/peerj-cs.2816

Sasi T, Lashkari AH, Lu R, Xiong P, Iqbal S (2024) A comprehensive survey on IoT attacks: Taxonomy, detection mechanisms and challenges. J Inf Intell 2(6):455–513. https://doi.org/10.1016/j.jiixd.2023.12.001

Seoane V, Garcia-Rubio C, Almenares F, Campo C (2021) Performance evaluation of CoAP and MQTT with security support for IoT environments. Comput Netw 197:108338. https://doi.org/10.1016/j.comnet.2021.108338

Siddharthan H, Deepa T, Chandhar P (2022) Senmqtt-set: An intelligent intrusion detection in IoT-MQTT networks using ensemble multi cascade features. IEEE Access 10:33095–33110. https://doi.org/10.1109/access.2022.3161566

Voloshyn V, Khan MS, Srivastava G, M, D (2024) Analysis of NIST lightweight cryptographic algorithms performance in IoT security environments based on MQTT. In: 2024 IEEE Wireless Communications and Networking Conference (WCNC), Dubai, UAE, pp 1–6 https://doi.org/10.1109/wcnc57260.2024.10571199

Yusoff ZYM, Ishak MK, Rahim LAB, Asaari MSM (2024) Improving smart home security via MQTT: Maximizing data privacy and device authentication using elliptic curve cryptography. Comput Syst Sci Eng 48(6):1669–1697. https://doi.org/10.32604/csse.2024.056741

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.